Powered by mindgrasp

Note 1

Java Overview and Core Features

- **Java**: High-level, platform-independent, object-oriented programming language.
- **History**:
- Developed in 1991 and released in 1995 by Sun Microsystems.
- Oracle acquired Sun Microsystems in 2010; Oracle and other vendors maintain Java.
- Releases range from Java 1.0 to modern versions such as Java 21; LTS versions receive extended support.
- **Uses**: Web development, Android apps, enterprise software, and games.
- **Platform independence**:
- Java source compiles into platform-independent bytecode in `.class` files.
- A JVM for each operating system runs the bytecode, enabling WORA: “Write Once, Run Anywhere.”
- *Example:* An application built on Windows servers can run on Linux servers without modification.
- **Other features**:
- **Object-oriented**: Organizes software into objects that combine data and behavior.
- **Robust and secure**: Uses garbage collection, memory management, and runtime error checking.
- **High performance**: Uses Just-In-Time (JIT) compilation.
- **OOP concepts**:
- **Class**: Blueprint defining objects’ properties and behaviors.
- **Object**: Class instance containing data and methods.
- **Inheritance**: One class acquires properties and behaviors from another.
- **Polymorphism**: Different objects respond to the same method in different ways.
- **Abstraction**: Hides implementation details and exposes essential features.
- **Encapsulation**: Bundles data and methods while restricting access to internal state.
- **Industry example**: Garbage collection and runtime checks support reliable applications handling millions of daily transactions, such as banking systems.

Note 2

Writing, Compiling, and Running a Java Program

- **Prerequisites**: Install Java; the lesson uses Notepad++ to write source code.
- **Program structure**:
- **Class declaration**: `public class HelloWorld` defines a class named `HelloWorld`.
- **Class body**: Statements, methods, and logic go between `{` and `}`.
- **Main method**: `public static void main(String[] args)` is the application entry point.
- **Main method components**:
- **`public`**: Allows the JVM to call the method.
- **`static`**: Lets the JVM call it without creating an object; `public` and `static` may appear in either order.
- **`void`**: The method returns no value.
- **`main`**: The method name recognized by the JVM.
- **`String[] args`**: Receives command-line arguments as an array of strings.
- **Printing output**:

- `**System**`: Predefined Java class.
- `**out**`: Public static `PrintStream` field of `System`.
- `**println()**`: Prints its argument and adds a newline.
- *Example:* `System.out.println("Welcome to the Kodewala!");`
- **Compile and run**:
1. Open a terminal in the source-file directory.
1. Compile with `javac HelloWorld.java`; this creates `HelloWorld.class` containing bytecode.
1. Run with `java HelloWorld`; the output is `Welcome to the Kodewala!`.
- **Naming rule**: The source filename must match the class name; otherwise, compilation produces an error.

Note 3

JDK, JRE, and JVM

- **JDK (Java Development Kit)**: Toolkit for developing Java applications.
- Includes the JRE, compiler `javac`, JVM, standard libraries, and development tools.
- **Tools**: `javap` disassembles, `javadoc` generates documentation, and `jdb` debugs.
- **Purpose**: Write, compile, and debug Java programs; required for development.
- **JRE (Java Runtime Environment)**: Environment for running Java applications without development tools.
- Includes the JVM, core Java libraries, and platform-specific native libraries.
- Suitable when a system only needs to run applications.
- **JVM (Java Virtual Machine)**: Executes Java bytecode and manages runtime memory.
- Converts bytecode into a machine-understandable format; allocates and releases memory.
- Starts when the `java` command runs and is destroyed after program execution.
- **Relationship**:
- **JDK** contains the JRE; **JRE** contains the JVM.
- A JRE is sufficient for running applications; a JDK is needed to develop them.

Note 4

Java Command-Line Arguments

- **Command-line argument**: Value supplied when launching a Java program, such as `java HelloWorld arg1 arg2`.
- **How arguments work**:
- Supply values separated by spaces after the class name.
- The JVM places them in the `String[] args` parameter of `main()`.
- All inputs arrive as strings; convert them to types such as `int`, `double`, `float`, or `char` when needed.
- **Accessing arguments**:
- Use indexes such as `args[0]`, `args[1]`, and `args[2]`.
- Check the number supplied with the array's `.length` property.
- *Example:* `java <class name> <arg0> <arg1> <arg2>` passes three arguments.
- **Why use them**: Make programs flexible by allowing inputs to change at launch.
- *Example:* Pass a filename or file path to a file-reading utility.
- *Example:* Pass values to a simple calculator.
- **Input validation**: Check that required arguments were supplied; otherwise, warn the user,

such as “Please enter the input.”

Note 5

Java Identifiers and Naming Conventions

- **Identifier**: Name assigned to a variable, class, method, package, interface, or other program element.
- *Example:* In the sample program, `Kodewala`, `main`, `String`, `args`, `balance`, and `location` are identifiers.
- **Rules**:
- **Allowed characters**: Letters, digits, `$`, and `_`; no other special characters.
- **First character**: Cannot be a digit.
- **Case sensitivity**: `Example` and `example` are different identifiers.
- **Length**: No maximum length; 4–15 characters is recommended for readability.
- **Reserved words**: Cannot be identifiers; for example, `for` is reserved.
- **Examples**:
- **Valid**: `FirstName`, `FIRSTNAME`, `firstname`, `y1`, `$firstname`, `total_sum`, `code123`.
- **Invalid**: `Test Name` contains a space; `34kodewala` starts with a digit.
- **Invalid**: `ca+t` contains `+`; `variable-5` contains `-`; `last_&_first` contains `&`.
- **Readable naming**:
- Choose meaningful, self-explanatory names to improve readability, debugging, and maintenance.
- *Example:* `FundTransfer` clearly describes a class that handles fund transfers.
- *Example:* `transfer(fromAccount, toAccount, amount)` makes the action and parameters clear.

Note 6

Java Data Type Categories

- **Strong typing**: Every Java value has a predefined data type.
- **Primitive types**: Built-in types include `boolean`, `char`, `byte`, `short`, `int`, `long`, `float`, and `double`.
- Choose a type with appropriate memory capacity and value range.
- *Example:* When choosing between `int` and `long` for large numerical identifiers, consider whether values may exceed `int`’s range.
- **Boolean size**: JVM-dependent; many JVMs use 8 bits.
- **Non-primitive types**: Reference types include built-in types such as `String` and arrays, plus user-defined classes such as `Person` and `Account`.
- Store references to data rather than the data itself.
- Include classes, arrays, interfaces, and enums; they have no fixed size and may grow dynamically.
- Default value is `null` when a reference variable is not initialized.
- **Safety**: Validate references before accessing them to avoid `NullPointerException`.

Note 7

Java Variables and Variable Types

- **Variable**: Named memory location that stores a value.
- **Data type**: Specifies the kind of value, such as `int`, `boolean`, `String`, `float`, or `char`.
- **Name**: Identifies the variable within its scope.
- **Value**: Data assigned to the variable.
- **Declaration**: A variable can be declared and assigned a value together, or declared first and assigned later.
- **Types of variables**:
- **Local variable**: Declared inside a method, block, or constructor.
- Scope is limited to that method, block, or constructor.
- Must be initialized before use; otherwise, compilation fails.
- **Instance variable**: Declared in a class, outside methods, blocks, and constructors.
- Created when an object is created and destroyed with that object.
- Receives a type-based default value if not explicitly initialized; an `int` defaults to `0`.
- Accessed through an object; can be initialized in a constructor or initialization block.
- **Static variable**: Listed as a variable category; further details are not provided in the material.

Note 8

Static Variables

- **Static variable**: Class-level variable declared inside a class, outside methods and constructors, using `static`.
- **Storage and lifetime**:
- One shared copy exists for the entire class, regardless of the number of objects.
- Created when the program starts and destroyed when it ends.
- **Declaration and initialization**:
- Cannot be declared inside an instance method.
- Can be initialized using a static block.
- **Access**:
- Within its class, the class name may be omitted; the compiler supplies it.
- From another class, use the class name to avoid ambiguity when names are shared.
- **When to use**:
- **Instance variables**: Store values unique to each object, such as each e-commerce user's shopping-cart items and total price.
- **Static variables**: Hold shared resources or constants, such as a counter of active users or a company code shared by employees.
- **Memory**: Using instance variables for shared data can increase memory consumption.
- **Common applications**:
- **Bank accounts**: Each account has an instance balance; all accounts share one static interest rate.
- **Object counting**: Increment a static counter when each object is created.
- **Student records**: Keep `name`, `rollNumber`, and `marks` as instance variables; use static `schoolName` for all students.

Note 9

Java Variable Scope

- **Variable categories**:
- **Member variables**: Instance and static variables declared inside a class but outside methods.
- **Local variables**: Variables declared within a method.
- **Member-variable scope**:
- Can be declared anywhere in the class outside methods.
- Accessible directly from member methods throughout the class.
- Access modifiers do not change their scope within the declaring class.
- Access from outside the class depends on access rules.
- **Local-variable scope and lifetime**:
- Available only within the method where declared; inaccessible outside it.
- Cease to exist when the method finishes.
- Curly braces `{ }` generally define a scope.
- **Practical uses**:
- **Member variables**: Represent persistent object state, configuration, or user details.
- *Example:* A `BankAccount` stores `accountNumber` and `balance` for use across transaction methods.
- **Local variables**: Hold temporary calculations or method-specific data.
- *Example:* A payment method stores a transaction ID during an operation.
- **Practice applications**:
- Create an `Employee` with member variables `name`, `id`, and `salary`, plus methods to update and display them.
- Calculate a two-number sum using local variables; use local variables for a car discount rate and discounted price.

Note 10

Java Assignment, Relational, and Logical Operators

- **Operator**: Special symbol that performs an operation on values or variables.
- **Assignment operator**:
- `=` assigns the right-hand value to the variable on the left.
- **Form**: `variable = value;`
- **Relational operators**:
- Compare values and return a boolean result; often used in loops and `if-else` conditions.
- **Operators**:
- `==`: equal to; `!=`: not equal to.
- `<`: less than; `<=`: less than or equal to.
- `>`: greater than; `>=`: greater than or equal to.
- **Logical operators**:
- Combine or invert boolean conditions; short-circuit evaluation may skip the second condition.
- **Operators**:
- `&&`: true only when both conditions are true; skips the second if the first is false.
- `||`: true when at least one condition is true; can skip the second if the first is true.
- `!`: reverses a condition's truth value.
- **Applications and practice**:
- Validate age with `age > 18 && age < 60`; check access with `isUserAdmin || isUserManager`.

- Write checks for positive-and-even numbers, divisibility by 3 and 5, or values within 10–50.
- A driving-eligibility example uses `age > 18 && age < 70`.
- Temperature example: identify days above 30°C when humidity is below 50%.

Note 11

Interactive Input with Scanner

- **Scanner**: Java class for reading input dynamically while a program runs, unlike command-line arguments supplied upfront.
- **Capabilities**:
- Reads from sources such as the console and files.
- Supports types including integers, floating-point numbers, and strings.
- Provides methods such as `nextInt()`, `nextLine()`, `next()`, and `nextDouble()`.
- **Setup and use**:

1. Import with `import java.util.Scanner;`.
1. Create a scanner connected to standard input: `Scanner scanner = new Scanner(System.in);`.
1. Prompt the user and call a method matching the expected input type.

- **Input validation**:
- Use `hasNextInt()` to check whether the next token is a valid integer.
- If validation fails, prompt the user to enter the value again.
- Validation helps prevent incorrect records and errors, such as accepting text instead of an age.
- **Practice**: Read and display an age (`int`), name (`String`), and salary (`double`), then add validation for those inputs.

Note 12

Java Methods and Their Declarations

- **Method**: Named block of code that performs a task, supports reuse, and exposes object behavior; every Java method belongs to a class.
- **General syntax**: `<access_modifier> <return_type> <method_name>(parameters) { body }`.
- **Declaration elements**:
- **Access modifier**: Controls visibility, such as `public`, `private`, `protected`, or default.
- **Return type**: Specifies the returned value's type; use `void` when nothing is returned.
- **Method name**: Unique identifier, typically written in camelCase.
- **Parameter list**: Optional typed inputs enclosed in parentheses.
- **Exception list**: Optional declaration of exceptions using `throws`.
- **Method body**: Curly-braced block containing the logic; a method may return a value.
- **Method categories**:
- **Built-in methods**: Supplied by Java class libraries; examples include `random()` and `parseInt()`.
- **User-defined methods**: Written by developers and adapted to requirements; examples include `sayHello()` and `addTwoNumbers()`.
- **Method forms**:
- **Instance method**: Defined in a class and accesses instance data through an object reference.

- **Static method**: Declared with `static` and accesses static data through the class name.
- **Method signature**:
- Consists of the method name and parameter list: parameter count, types, and order.
- Return type and exceptions are not part of the signature.
- *Example:* `int addTwoNumbers(int a, int b)` has name `addTwoNumbers` and two `int` parameters.

Note 13

Calling Methods, Naming Conventions, and Benefits

- **Naming conventions**:
- Start with a lowercase verb; use camelCase for multiword names, capitalizing later words.
- *Examples:* `addTwoNumbers`, `sayHello`, `doSomething`, and `printMessage`.
- Methods may share a name through overloading when their parameter lists differ.
- **Calling methods**:
- Instance methods are called through an object reference.
- *Example:* Create a `Kodewala` object named `obj`, then call `obj.displayMessage()`.
- A method returns control after its body completes, a `return` executes, or an exception is thrown.
- **Benefits**:
- **Reusability**: Write logic once and use it repeatedly.
- **Abstraction and encapsulation**: Hide implementation details behind a simpler interface.
- **Modularity**: Divide programs into manageable units that are easier to understand, maintain, and debug.
- **Customization**: Adapt methods to specific tasks; clear organization can support optimization.
- **Application examples**:
- Reuse `calculateDeliveryFee(double distance)` for food orders.
- Implement shopping-cart totals, `sendEmail(String recipient, String message)`, or `validateInput(String input)`.
- Build calculator methods for basic operations, banking methods such as `deposit(double amount)` and `checkBalance()`, or `trackOrder(int orderId)`.

Note 14

Java Access Modifiers: Default and Private

- **Access modifier**: Controls the visibility of classes, constructors, variables, and methods to protect data and regulate access.
- **Default access**:
- No keyword is written; access is limited to the same package.
- *Example:* `Customer` and `Order` in the same package can access default `customerName` and `sayHello()` members.
- Accessing a default-access class or member from another package causes a compile-time error.
- **Private access**:
- Declared with `private`; accessible only within the declaring class.
- Other classes cannot directly access private members, even within the same package.

- *Example:* A `BankAccount` can keep `balance` and `updateBalance()` private while exposing a public `deposit()` method for controlled updates.
- Directly accessing `balance` or calling `updateBalance()` from `Main` causes a compile-time error.

Note 15

Java Protected Access Modifier

- **Protected access**: Declared with the `protected` keyword; controls access to methods and variables.
- **Same-package access**: Protected members are accessible to classes in the same package.
- **Subclass access**: Subclasses in different packages can access protected members through inheritance.
- **Example setup**: Define a base `Employee` class in `com.kodewala.example` and a subclass in `com.kodewala.client` to demonstrate cross-package access.

Note 16

Java Access Modifiers

- **Public**: Accessible from anywhere in the program.
- **Package access**:
- **Private**: Restricts access to the declaring class; useful for protecting sensitive data.
- *Example:* A bank account balance is private and changed through validated `deposit` or `withdraw` methods.
- **Default**: Accessible only to classes in the same package.
- *Example:* Inventory helper methods can be shared by `InventoryManager` and `OrderProcessor` in one package.
- **Protected**: Supports access within the package and by subclasses, including subclasses in other packages.
- *Example:* A payroll bonus method can let `Manager` and `Developer` subclasses implement custom rules.
- **Public use case**: Hotel APIs such as `searchHotel()` and `bookHotel()` can be available to all users.
- **Key distinction**: Default access does not extend to subclasses in other packages; protected access does.

Note 17

Calling Methods in Java

- **Method**: Group of statements that performs a task and may return a result; calling methods supports code reuse and organization.
- **Instance method**:
- Belongs to an object and is called through an instance.
- *Example:* A method that updates a particular account balance.

- **Static method**:
- Belongs to the class and is called with `ClassName.methodName(args)`.
- Does not require creating an object.
- *Example:* `MathOperations.multiplyNumbers(10, 5)` calls a class method to multiply two numbers.
- **Choosing a method**:
- Use static methods for tasks independent of object-specific data.
- Use instance methods when the task uses or changes data belonging to a specific object.
- **Industry use**: Methods divide complex work into manageable, reusable, modular operations, including calculations and data processing.

Note 18

Static and Instance Method Behavior

- **Static methods**:
- Belong to the class, use the `static` keyword, and can be called without an object.
- *Example:* `TaxCalculator.calculateTax()` or a shared bank-interest calculation.
- Common uses include utilities such as tax calculations and logging.
- Resolved through static binding; they cannot be overridden.
- A same-named subclass static method hides the superclass method rather than overriding it.
- **Instance methods**:
- Belong to a specific object and require an instance for a call.
- *Example:* A `BankAccount` deposit method updates that account's balance.
- Use `this` to refer to the current object; dynamic binding selects overridable methods at runtime.
- Common uses include processing an order or changing account-specific data.
- **Memory notes**:
- Static and instance methods are associated with class memory; local variables and arguments are stored on the stack.
- The material describes the pre-Java 8 method area as Permanent Generation and its replacement as Metaspace from Java 8.

Note 19

Java Conditional Statements

- **`if`**: Executes its block when a condition is true; otherwise skips it.

1. Check the condition.
1. Run the block if true; skip it if false.
1. Continue after the block.

- **`if-else`**: Chooses between two blocks; the `else` block runs when the condition is false.
- *Example:* Check whether a number is positive, then print the appropriate result.
- **`if-else-if` ladder**: Tests conditions in order and executes the first matching block; final `else` handles no match.
- *Example:* An online store can give orders above ₹10,000 a 20% discount and orders below ₹1,000 no discount.
- **Common uses**: Access permissions, user-input handling, calculations, and business

decisions.
- **Practice applications**:
- Test even or odd with `number % 2 == 0`.
- Set shipping to free above ₹5,000, ₹100 for ₹2,000–₹5,000, and ₹200 below ₹2,000.

Note 20

Java For Loops and Nested Loops

- **`for` loop**: Repeats a block while its condition remains true; useful when the iteration count or range is known.
- **Loop structure**:
1. **Initialization**: Runs once to set the loop variable.
1. **Condition**: Checked before each iteration; a false result ends the loop.
1. **Update**: Runs after each iteration to change the loop variable.
- **Execution example**: Starting `i` at 1 and checking `i <= 10` prints 1–10; the loop ends when `i` reaches 11.
- **Nested loops**: An inner loop completes its work for each outer-loop iteration.
- *Example:* The outer loop controls star-pattern rows; the inner loop prints an increasing number of stars per row.
- **Applications and practice**:
- Process many transactions or account records; loops also support arrays, collections, strings, and algorithms.
- Practice printing 1–10, summing even array values, or counting even and odd values.

Note 21

Java While and Do-While Loops

- **`while` loop**: Checks its condition before each iteration, so it may execute zero times.
1. Initialize the control variable.
1. Check the condition; run the body when true.
1. Update the variable and repeat until the condition is false.
- *Example:* Start `i` at 1, print and increment while `i <= 5`.
- **Condition-controlled use**: Choose `while` when the number of iterations is unknown and repetition depends on an event.
- *Example:* A guessing game repeats input until the user guesses 5, then exits.
- *Example:* Keep checking a bank balance until it drops below a threshold.
- **`do-while` loop**: Checks the condition after the body, guaranteeing at least one execution.
- Use it when an action must happen before the continuation condition is checked.
- **Choosing a loop**:
- Use `for` when the repetition count is known, such as printing 1–5.
- Use `while` when repetition continues until a condition changes, such as a correct guess.
- **Practice**: Print 1–5 with `while`; keep requesting input until a number greater than 10 is entered.

Note 22

Java Switch Statements

- **`switch`**: Selects a code block by comparing an expression with predefined case values; often clearer than a long `if-else-if` ladder.
- **Supported values**: `byte`, `short`, `char`, `int`, `long`, `String`, enums, and corresponding wrapper classes such as `Integer`.
- **Structure and behavior**:
- **`case`**: Runs when its value matches the expression.
- **`break`**: Exits the switch; without it, execution continues into following cases (fall-through).
- **`default`**: Optional block for values with no matching case.
- **Example**: If `day` equals 3, `case 3` prints “Wednesday”; if no case matches, `default` can print “Invalid day.”
- **Applications and practice**:
- A billing system can select a discount based on Silver, Gold, or Platinum membership.
- Map inputs 1–7 to weekday names, reject invalid values, or map a month number to its name.

Note 23

Java break Statement

- **Purpose**: Immediately exits the enclosing loop or `switch` case when execution reaches `break`.
- **Loop behavior**:
1. **Encounter**: The statement runs inside the loop, often when a condition is met.
1. **Exit**: The loop terminates immediately, regardless of remaining iterations.
1. **Continue after loop**: Control moves to the first statement following the loop.
- **Syntax**:
- `for (initialization; condition; update) { if (conditionToBreak) { break; } }`
- **Applications and practice**:
- **Search**: Find `"apple16"` in a product array, display a message, and stop at its first occurrence.
- **Industry example**: Stop searching a large e-commerce inventory once the required product is found.
- **Practice**: Print numbers 1–10, stopping when the number reaches 5; search an array for a number and report whether it was found.

Note 24

Java continue Statement

- **Purpose**: Skips the current loop iteration and proceeds to the next one; usable in `for`, `while`, and `do-while` loops.
- **Loop behavior**:
1. **Encounter**: Usually reached inside a conditional block in the loop body.
1. **Skip**: Remaining statements in the current iteration are bypassed.
1. **Next iteration**: The loop proceeds, checking its condition again and performing the `for` update when applicable.
- **Syntax**:
- `for (initialization; condition; update) { if (conditionToSkip) { continue; } }`

- **Applications and practice**:
- **Product counting**: Skip products whose names start with `"apple"`; count the other products.
- **Industry example**: Skip invalid or empty customer-order records so only valid data is processed.
- **Practice**: Skip multiples of 3 when printing 1–10; skip student grades below 50%.

Note 25

Java Array Fundamentals and Operations

- **Array**: Object that stores a fixed number of elements of the same data type, such as `int` or `String`.
- **Properties and trade-offs**:
- **Indexing**: Starts at 0; elements are accessed by position.
- **Storage and size**: Elements occupy a continuous memory block; size is fixed when the array is created.
- **Benefits**: Efficient storage, retrieval, sorting, and random access by index.
- **Limitation**: An array cannot grow dynamically at runtime.
- **Declaration and creation**:
- **Declaration**: `int[] numbers;` or `int numbers[];`; the data type determines element types.
- **Allocation**: `int[] numbers = new int[10];` creates space for 10 integers.
- **Direct initialization**: `int[] numbers = {5, 10, 15, 20, 25};`
- **Dynamic initialization**: `int[] evenNumbers = new int[5];` creates five elements, initially zero for `int`.
- **Working with elements**:
- **Assign**: `numbers[0] = 5;` stores a value at index 0.
- **Read**: `numbers[0]` accesses the first element; for example, `numbers[2]` reads the third.
- **Change**: `numbers[0] = 50;` replaces the value at index 0.
- **Length**: `numbers.length` returns the number of elements; a five-element array has length 5.
- **Index and default-value rules**:
- **Valid range**: For size 5, valid indices are 0–4.
- **Out of bounds**: Accessing `numbers[5]` throws `ArrayIndexOutOfBoundsException`.
- **Defaults**: Unassigned `int` elements contain `0`.

Note 26

Java Array Iteration and Practical Uses

- **Reading elements**:
- **By index**: Use the array name and index to retrieve a particular element, such as indices 0 and 4.
- **By loop**: Iterate through the array to access its elements in sequence.
- **Searching an array**:
- **Example**: Store four employee names, then loop through them to check whether a user-entered name exists.
- **Product search**: A loop can inspect products and stop when the target is found.
- **Working with related arrays**:

- **Example**: Store electronic-item names and prices in separate arrays; find and print the product with the highest price.
- **Practice**: Keep book titles and prices in separate arrays, then print each title with its price.
- **Industry use**: Arrays can hold product inventory and support quick lookups when customers add items to a cart.
- **Common questions and tasks**:
- **Concepts**: Declaration, initialization, default values, length, bounds errors, and whether arrays can hold different types.
- **Comparisons and methods**: Array versus `ArrayList`, `Arrays.sort()`, and reversing an array.
- **Practice**: Create, initialize, and print electronic-product items.

Note 27

Java String Immutability and Creation

- **String**: Object representing a sequence of characters.
- **Immutability**: Once created, a `String` object's content cannot change; a modification produces another object.
- **Creation methods**:
- **Literal**: `String s1 = "Kodewala";` uses the String Constant Pool (SCP).
- **`new` keyword**: `String s2 = new String("Kodewala");` explicitly creates a heap object.
- **Why immutability matters**:
- **Efficiency**: Shared strings cannot be unexpectedly altered through another reference.
- **Security and thread safety**: Immutable content avoids changes that could affect other users of the same object.
- **Example**: If `s1` and `s2` share a pooled string, changing the shared object would affect both; immutability prevents this.
- **Modification example**:
- **Concatenation**: `String s2 = s1.concat(" Academy");` creates a new string rather than changing `s1`.

Note 28

Java String Constant Pool and Heap Objects

- **String Constant Pool (SCP)**:
- **Literal reuse**: Java checks whether a matching literal exists in the pool; if so, references share that object.
- **Example**: `String s1 = "Kodewala"; String s2 = "Kodewala"; String s3 = "Kodewala";` uses one pooled object and three references.
- **Objects created with `new`**:
- **Heap allocation**: Each `new String("Kodewala")` call creates a distinct heap object, even when contents match.
- **Pool literal**: The literal is also placed in the SCP if it is not already there.
- **Example**: Two `new String("Kodewala")` calls create two heap objects; if the literal was initially absent, they also use one pooled object.
- **Object-count examples**:
- **Two `new` calls**: Three objects total if the literal was not already pooled—one SCP object and two heap objects.

- **Two literals and two `new` calls**: Three objects total—one shared SCP object and two distinct heap objects.
- **Comparing strings**:
- `==`: Compares object references, not string contents; two distinct heap objects compare false.
- `.equals()`: Compares string contents; equal text can return true even for distinct objects.
- **Memory guidance**: Prefer literals for common strings to reuse pooled objects and avoid unnecessary heap allocations.

Note 29

Java String Immutability

- **Immutability**: A `String` cannot be changed after creation; operations that appear to modify it create another string.
- **Concatenation**:
- **`concat()`**: Creates a new string rather than changing the original.
- *Example:* If `s1` refers to `"Kodewala"`, `s1.concat(" Academy")` produces `"Kodewala Academy"`; `s1` still refers to `"Kodewala"`.
- **References**:
- **`s1`**: Refers to the original literal in the String Constant Pool (SCP).
- **`s2`**: May refer to the new concatenated string on the heap.
- `==`: Compares references, so different objects yield `false`; `.equals()` compares string contents.
- **Reasons for immutability**:
- **Security**: Prevents sensitive string values from being altered unexpectedly.
- **Sharing and performance**: Enables reference reuse and JVM memory optimizations.
- **Thread safety**: Immutable values can be shared without mutation-related data corruption.
- **Hashing and pooling**: Supports stable hash codes and protects pooled literals from changes.

Note 30

String Interning and the String Constant Pool

- **String interning**: Reuses one pooled reference for a given string value, reducing duplicate literal storage.
- **Literal handling**:
- **New literal**: Added to the SCP if that value is not already present.
- **Existing literal**: Java reuses the reference already in the SCP.
- **`intern()`**: Returns the SCP reference for a string value, adding it to the pool if needed; it does not move or replace the original heap object.
- **Object example**:
- **`String s1 = "Kodewala Academy"`**: Refers to the pooled literal.
- **`String s2 = new String("Kodewala Academy")`**: Creates a separate heap object; the literal is also pooled.
- **`String s3 = s2.intern()`**: Refers to the pooled string, so `s1` and `s3` share a reference while `s2` remains separate.
- **Comparison practice**: Compare a literal and `new String()` using both `==` and `.equals()`

to distinguish reference identity from content equality.

Note 31

Common Java String Methods

- **Length and character access**:
- **`length()`**: Returns the number of characters; useful for validating input length, such as password requirements.
- **`charAt(index)`**: Returns the character at the specified index; can be used to print or inspect characters in a loop.
- **Comparison**:
- **`equals(obj)`**: Compares string contents.
- **`equalsIgnoreCase(str)`**: Compares contents without distinguishing letter case; useful for usernames or search terms.
- **Searching and extraction**:
- **`indexOf(str)`**: Finds the first occurrence index; commonly used to locate a character or substring.
- **`lastIndexOf(str)`**: Finds the last occurrence index.
- **`substring(begin)`**: Returns characters from the start index to the end.
- **`substring(begin, end)`**: Uses an inclusive start index and exclusive end index; useful for extracting data from emails or logs.
- **Modification and splitting**:
- **`toUpperCase()` / `toLowerCase()`**: Change letter case for input standardization, such as email normalization.
- **`trim()`**: Removes leading and trailing whitespace.
- **`replace(oldChar, newChar)`**: Replaces every occurrence of a character; for example, spaces with underscores or hyphens.
- **`split(regex)`**: Splits a string into an array using a regular-expression delimiter; useful for CSV fields or sentence tokens.
- **Content checks and conversion**:
- **`contains(sequence)`**: Checks whether a character sequence occurs in the string.
- **`startsWith(prefix)` / `endsWith(suffix)`**: Check the beginning or end, such as validating `IMG_` and `.jpg` filenames.
- **`toCharArray()`**: Converts the string to a character array; can support tasks such as counting vowels.

Note 32

String Concatenation and Practical Use

- **Concatenation**: Joins two or more strings to form a combined string.
- **Common approaches**:
- **`+` operator**: Simple, commonly used concatenation syntax.
- **`concat()`**: Appends a specified string to the current string and returns a result; the original `String` remains unchanged.
- **Practical uses**:
- **File paths and messages**: Append fixed text to create paths or dynamic messages.
- **Data processing**: Manipulate text while reading or writing CSV and JSON files, or cleaning

data.
- **User input**: Store names, addresses, and other entered values.
- **Practice tasks**:
- **Character and substring work**: Print characters with `charAt()` and extract an email domain with `substring()`.
- **Transformations**: Replace spaces, split comma-separated values, and concatenate multiple strings.
- **Validation**: Check for words, count vowels after `toCharArray()`, and validate filename prefixes or suffixes.

Note 33

Java StringBuffer

- **`StringBuffer`**: Mutable character sequence whose contents can be changed without creating a new object for each modification.
- **Thread safety and performance**:
- **Synchronization**: Its methods are synchronized, making shared access thread-safe.
- **Trade-off**: Synchronization overhead can make it slower than `StringBuilder`, especially in single-threaded use.
- **Comparison with `String`**: `String` is immutable and therefore safe to share; `StringBuffer` is useful for repeated changes.
- **Common operations**:
- **`append()`**: Adds data to the end of the buffer.
- **`insert()`**: Adds text at a specified position.
- **`delete()`**: Removes a portion of the buffer.
- *Example:* A to-do list can add tasks, insert or update task text, and remove entries using these operations.
- **Capacity**:
- **Default initial capacity**: 16 characters when no initial string or custom capacity is provided.
- **Growth**: Allocates larger internal storage when content exceeds capacity.
- **Formula**: `new capacity = (current capacity × 2) + 2`.
- **Practical use**: Efficiently update dynamic text, such as text displayed in a chat interface.

Note 34

Java StringBuilder

- **`StringBuilder`**: Mutable character sequence that supports repeated changes without creating a new string object for each operation.
- **Concurrency and speed**:
- **Synchronization**: Methods are not synchronized, so `StringBuilder` is not thread-safe for concurrent access to the same object.
- **Performance**: Avoiding synchronization overhead generally makes it faster than `StringBuffer` in single-threaded scenarios.
- **Choice**: Prefer `StringBuffer` when synchronized access is needed; use `StringBuilder` for non-concurrent modifications.
- **Dynamic construction**:
- **SQL queries**: Append conditions to a base query according to user-provided filters.

- *Example*: Starting with `SELECT * FROM employees WHERE 1=1`, filters for department `IT`, salary above `50000`, and joining date from `2020-01-01` produce `AND department = 'IT' AND salary > 50000 AND joining_date >= '2020-01-01'`.
- **Web or app responses**: Build API or webpage responses dynamically.
- **Capacity topics**: The material identifies default capacity and capacity changes as topics to understand when constructing strings.

Note 35

Object-Oriented Programming and Encapsulation

- **Object-oriented programming (OOP)**: Organizes programs around objects that model real-world entities; supports modular, reusable, maintainable code.
- **Encapsulation**: Combines data and methods in a class while restricting direct access to internal data.
- **Implementation**:
- **Private fields**: Hide class variables from direct access by other classes.
- **Public methods**: Getters and setters provide controlled reading or modification.
- **Controlled operations**: Methods can validate requests and preserve data integrity.
- **Bank account example**:
- **Hidden balance**: A user cannot directly set the account balance.
- **Allowed actions**: Deposit and withdrawal methods control balance changes.
- **Safeguards**: Authentication and withdrawal limits can restrict operations.
- **Phone analogy**:
- **Private contents**: The phone's internal data is hidden.
- **Public access**: Others may use permitted functions such as calls, music, or maps.
- **Security**: A PIN protects access to private details.

Note 36

Encapsulation in Java

- **Encapsulation**: Hides sensitive data and provides controlled access through public methods.
- **ATM example**:
- **Private fields**: `balance` and `pin` cannot be accessed or changed directly outside the class.
- **Public methods**:
- **`withdraw(amount, enteredPin)`**: Checks the PIN and available balance before withdrawing.
- **`deposit(amount, enteredPin)`**: Verifies the PIN before adding money to the balance.
- **Benefits**:
- **Security and integrity**: Prevents unauthorized or accidental changes.
- **Maintainability**: Controlled updates make code easier to maintain and scale.
- **Practice applications**:
- *Example:* Build an ATM with private `balance` and `pin`, exposing only deposit and withdrawal methods.
- *Example:* Create a `BankAccount` with private fields, controlled getters/setters, and security validations.

Note 37

Java Constructors and Initialization

- **Constructor**: Special class-named block with no return type; initializes an object when it is created.
- **Invocation and initialization**:
- **Automatic call**: Creating an object with `new` invokes a constructor.
- **Example:** `Account account = new Account();` runs `Account()` and its body.
- **Purpose**: Sets initial attribute values so objects begin in a valid state.
- **Default constructor**:
- **Compiler-provided**: Java supplies a no-argument constructor if none is declared.
- **Explicitly defined**: A developer may also write a no-argument constructor.
- **Superclass call**: The default constructor includes an implicit `super()` call.
- **Constructor rules**:
- **Name and return type**: Must match the class name and have no return type.
- **Modifiers**: Cannot be `abstract`, `final`, `static`, or `synchronized`; access modifiers control who can call it.
- **First statement**: Must call `super()` or `this()`; Java inserts no-argument `super()` if omitted.
- **Constructor versus method**:
- **Constructor**: Runs during object creation, has the class name, and has no return type.
- **Method**: Can have any valid name and a return type, including `void`; can be called repeatedly.
- **Superclass initialization**:
- **`super()`**: Calls the parent constructor; every Java class ultimately inherits from `Object`.
- **Inherited `Object` methods**: Include `toString()`, `equals()`, and `hashCode()`.

Note 38

Constructor Chaining in Java

- **Constructor chaining**: Calling one constructor from another to reuse initialization code and reduce duplication.
- **Forms of chaining**:

1. **Within the same class**: `this()` calls another constructor and must be the first statement.

- *Example:* A default `Database()` constructor delegates to a parameterized constructor, which initializes values and calls `connect()`.

1. **Between parent and child classes**: `super()` calls the parent constructor and must be the child constructor's first statement.

- *Example:* `Manager` calls `super(name)` so `Employee` initializes the manager's name first.
- **Benefits**:
- **Code reuse**: Avoids repeating initialization logic.
- **Reliable initialization**: Parent-class setup occurs before subclass-specific setup.
- **Applications and practice**:
- *Example:* Initialize a banking `Account` with default or user-specific details.
- **Exercise**: Create `Company` and `Employee` classes, using `super()` to initialize the company name.

Note 39

Java Inheritance and Its Benefits

- **Inheritance**: Lets a class reuse another class's fields and methods, then add or adapt features.
- **Core terms**:
- **Superclass or parent**: Class whose members are inherited.
- **Subclass or child**: Class that inherits and may define additional fields and methods.
- **Syntax**: Use `extends` to establish class inheritance.
- **Bank account example**:
- **`Account`**: Parent class with a balance and `showBalance()`.
- **`SavingAccount`**: Extends `Account`, adds `interestRate` and `calculateInterest()`.
- **Initialization**: `super(balance)` invokes the parent constructor.
- **Benefits and uses**:
- **Code reuse**: Reduces duplicated fields and methods.
- **Overriding and polymorphism**: Subclasses can provide specialized method implementations.
- **Abstraction and structure**: Supports common designs and clearer class hierarchies.
- *Example:* A banking system can share `showBalance()` through `Account`, with `SavingAccount` and `CurrentAccount` as subclasses.

Note 40

Types of Inheritance in Java

- **Inheritance structures**:

1. **Single**: One child inherits from one parent.
- *Example:* `SavingAccount` extends `Account`.
1. **Multilevel**: A child also acts as the parent of another class.
- *Example:* `Account` → `SavingAccount` → `PremiumSavingAccount`.
1. **Hierarchical**: Multiple child classes inherit from one parent.
- *Example:* `Vehicle` can have `Car` and `Bike` subclasses.
1. **Multiple**: A class inherits features from multiple parents; Java supports this through interfaces, not multiple classes.
1. **Hybrid**: Combines inheritance structures; Java can model such combinations using interfaces.

- **Class inheritance restriction**:
- **Ambiguity**: Two parent classes might define the same method, leaving the child's method call unclear.
- **Diamond problem**: A class inherits the same method through multiple parent paths.
- **Java's approach**: Disallows multiple inheritance with classes to prevent this ambiguity.

Note 41

Inheritance Conflicts and Child-Class Capabilities

- **Interface default-method conflicts**:
- **Java 8**: Default methods in interfaces can create conflicts resembling the diamond problem.
- **Resolution**: The implementing child class must override the conflicting method and explicitly choose its behavior.
- **What a child class can do**:
- **Use inherited members**: Access inherited fields and call inherited methods where permitted.
- **Add members**: Declare new fields and methods.
- **Override instance methods**: Provide a specialized implementation of a parent method.
- **Hide static methods**: Declare a static method with the same signature as a parent static method.
- **Initialize the parent**: Call its constructor implicitly or with `super()`.
- **Inheritance practice examples**:
- *Example:* `Employee` subclasses `Manager` and `Developer` override `calculateSalary()`.
- *Example:* `Product` subclasses `ElectronicsProduct` and `ClothingProduct` override `applyDiscount()`.
- *Example:* `Employee` → `SoftwareEngineer` → `SeniorSoftwareEngineer` forms multilevel inheritance.

Note 42

Abstract Classes in Java

- **Abstract class**: Class declared with `abstract`; serves as a blueprint and cannot be instantiated directly.
- **Methods and structure**:
- **Abstract methods**: Have no implementation; concrete subclasses must implement them.
- **Regular methods**: Have a body and can provide shared behavior.
- **Purpose**: Reuses common code and defines structure for related classes.
- **Bank account example**:
- **`Account`**: Abstract parent with a `balance` field and `showBalance()` method.
- **`calculateInterest()`**: Abstract method that subclasses must implement.
- **`SavingAccount`**: Extends `Account`, adds an interest rate, and implements interest calculation.
- **`CurrentAccount`**: Extends `Account` and implements interest calculation without adding interest.

Note 43

Abstract Classes and Abstraction in Java

- **Abstract class**: superclass blueprint that defines shared structure while leaving some behavior to subclasses.
- **Abstract methods**: declared without a body; subclasses must implement them.
- **Concrete methods**: include a body and can be inherited directly.
- **Instantiation**: cannot create objects directly from an abstract class.
- **Constructors**: allowed and called when a subclass object is created, initializing common properties.

- **Other features**: must be declared with `abstract`; may contain static methods and final methods.
- **Final methods**: prevent subclasses from changing their implementation.
- **Methods**: an abstract class may contain both abstract and concrete methods, or no abstract methods at all.
- **Abstraction**: hides implementation details and exposes only the functionality users need.
- **Purpose**: lets users focus on what an object does rather than how it works.
- *Example:* An ATM lets users withdraw cash, check balances, and transfer money without revealing backend transaction processing.
- *Example:* A car's steering wheel, accelerator, and brakes hide engine, fuel-injection, and braking mechanics.
- *Example:* A smartphone supports calls, messaging, and browsing without exposing operating-system or hardware interactions.
- **Using abstraction**: users work through a general type while implementation details remain in a specific subclass.
- **Bank example**: the `Bank` abstraction defines `loanInterestRate()`; `HDFC` contains the actual logic.
- **Benefit**: working through `Bank` exposes what the operation does, not its implementation.
- **When to use**: define a common structure for related classes while allowing subclasses to provide specific behavior.
- **Industry example**: Savings, Current, and Loan accounts share core banking behaviors but can implement them differently.
- **Design exercise**: an abstract `Vehicle` can declare `startEngine()`, implemented by `Car` and `Bike`.
- **Design exercise**: `BankAccount` can declare `calculateInterest()`, implemented differently by `SavingAccount` and `FixedDepositAccount`.

Note 44

Polymorphism and Dynamic Method Dispatch

- **Polymorphism**: "many forms"; the same method or reference can produce different behavior depending on context or actual object type.
- *Example:* One person may act as a Student at college, a Boyfriend in a relationship, and an Employee at work.
- **Two main types**:

1. **Compile-time polymorphism**: method overloading; Java selects a method using its arguments during compilation.
1. **Runtime polymorphism**: method overriding; Java selects the implementation using the actual object at runtime.

- **Runtime dispatch**:
- **Compile-time check**: compiler verifies the method exists in the reference type, such as `Person`.
- **Runtime execution**: actual object type, such as `Student` or `Boyfriend`, determines which overridden method runs.
- **Mechanism**: inheritance and a parent-class reference holding a child-class object enable dynamic method dispatch.
- **Payment example**: `Payment` defines `processPayment()`; `UPIPayment` and `CCPay` override it.
- **Selection**: a `Payment` reference can point to either child object; the matching child implementation runs at runtime.

- **Extensibility**: new options, such as NetBanking or wallets, can be added without changing existing core logic.
- **Benefits and trade-offs**:
- **Benefits**: code reuse, readability, maintainability, flexibility, and generic code that supports multiple object types.
- **Trade-offs**: behavior can be harder to understand in large systems; runtime selection may add processing overhead.
- **Industry examples**:
- **Payment processing**: overridden methods handle UPI and credit-card transactions according to the selected payment type.
- **Logging**: `Logger.log()` can be overridden by `ConsoleLogger`, `FileLogger`, and `DatabaseLogger`.
- **Use**: switch logging mechanisms dynamically without changing core logic.

Note 45

Method Overloading in Java

- **Method overloading**: multiple methods share a name but differ in parameter number, types, or order.
- **Aliases**: compile-time polymorphism, static polymorphism, and early binding.
- **Resolution**: Java chooses the matching method at compile time based on the supplied arguments.
- **Ways to overload**:
- **Number of parameters**: provide different argument counts.
- **Parameter types**: use different types, such as `String` and `int`.
- **Parameter order**: use the same types in a different order, such as `(String, int)` and `(int, String)`.
- *Example*: `doRegistration(String aadhar)`, `doRegistration(String mobile, int otp)`, `doRegistration(String name, String email)`, and `doRegistration(int otp, String mobile)`.
- **If no exact match exists**:
- **Widening within a type family**: Java may convert `int` to `long`.
- **Broader type**: if a same-family match is unavailable, Java may consider a wider type such as `float`.
- **Why it is useful**:
- **Readability**: one meaningful name represents related operations.
- **Maintainability and reuse**: reduces inconsistent naming and duplicated code.
- **Binding**: compile-time selection avoids runtime method-selection decisions.
- **Examples**:
- *Example:* `processPayment(String cardNumber)` handles cards; `processPayment(String upiId, double amount)` handles UPI; `processPayment(String walletId, int otp, double amount)` handles wallets.
- *Example:* A booking system can overload `book()` for a name, a name plus email, or a name plus phone number.

Note 46

Method Overriding and Its Rules

- **Method overriding**: a child class supplies a new implementation of an inherited parent method.
- **Signature**: child method uses the same name and parameters as the parent method.
- **Effect**: changes inherited behavior and supports runtime polymorphism.
- *Example:* `UPIPayment` overrides `Payment.processPayment()`; calling through a `Payment` reference runs the child implementation.
- **Requirements and restrictions**:
- **Inheritance**: required; overriding occurs between parent and child classes.
- **Access**: child method cannot be more restrictive than the parent method; `public` to `private` is invalid.
- **Static methods**: not overridden; redefining one in a subclass is method hiding.
- **Final methods**: cannot be overridden.
- **Constructors**: cannot be overridden because they are not inherited.
- **Return type**: may be covariant, returning a subclass of the parent method's return type.
- **Runtime behavior**: when a parent reference points to a child object, the actual object determines which overridden method executes.

Note 47

Java Interfaces

- **Interface**: contract describing what an implementing class must do, without specifying how it does it.
- **Implementation**: a class that implements an interface must provide implementations for its methods.
- **Purpose**: supports abstraction, multiple inheritance of type, and loose coupling between classes.
- **Interface members**:
- **Methods**: abstract by default in the material's pre-Java-8 description.
- **Variables**: `public`, `static`, and `final` by default; they are constants.
- **Example declaration**:
- **Interface**: `public interface KodewalaService` declares the service contract.
- **Constant**: `public static final String COMPANY = "KodewalaPay"` is accessible everywhere, belongs to the interface, and cannot be changed.
- **Abstract method**: `public abstract void provideService()` has no body and must be defined by implementing classes.

Note 48

Java Interfaces: Rules and Relationships

- **Interface**: contract declaring behaviors that implementing classes must provide.
- **Declaration and defaults**:
- **Keyword**: declare with `interface`; classes follow it with `implements`.
- **Methods**: implicitly `public` and `abstract`, except `default`, `static`, and `private` methods supported in later Java versions.
- **Fields**: implicitly `public`, `static`, and `final`; interfaces cannot have instance variables or constructors.
- **Relationships**:

- **Class extends class**: inherits features from another class.
- **Class implements interface**: follows its contract; must implement every method unless the class is abstract.
- **Interface extends interface**: may extend one or multiple interfaces.
- **Interface implements class**: not allowed; interfaces define behavior rather than class implementation.
- **Use and access**:
- **Instantiation**: an interface cannot be instantiated, but its reference can point to an implementing-class object.
- **Multiple inheritance**: classes can implement multiple interfaces, avoiding Java's restriction on extending multiple classes.
- **Loose coupling**: code depends on contracts rather than specific implementations, supporting flexibility and maintainability.

Note 49

Interface Methods and Examples in Java 8 and 9

- **Java 8 default methods**: use `default` to provide an implementation; adding one can preserve compatibility with existing implementations.
- **Java 8 static methods**:
- **Ownership**: belong to the interface and cannot be overridden by implementing classes.
- **Purpose**: provide interface-related utility behavior.
- **Java 9 private methods**:
- **Purpose**: share internal logic among default or static methods without duplicating code.
- **Visibility**: unavailable to implementing classes; improve encapsulation and readability.
- **Online payment example**:
- **Abstract `processPayment()`**: each payment type supplies its own payment logic.
- **Default `validateTransaction()`**: validates a transaction before payment.
- **Static `checkServiceStatus()`**: checks whether the payment service is active.
- **Private `log()`**: performs internal logging used by interface methods.

Note 50

BankAccount Interface and Account Implementations

- **BankAccount**: interface defining shared operations for different bank-account types.
- **Contract**:
- **`deposit(double amount)`**: adds money to the account.
- **`withdraw(double amount)`**: withdraws money subject to account rules.
- **`checkBalance()`**: returns the current balance.
- **`MIN_BALANCE`**: constant set to INR 1,000.00.
- **Account implementations**:
- **Savings account**: supports deposits and long-term saving; withdrawals must leave at least ₹1,000.
- **Current account**: designed for businesses; allows deposits and withdrawals without a minimum balance.
- **Benefits**:
- **Standardization and reuse**: accounts share a consistent set of operations, reducing

duplication and errors.
- **Scalability**: new account types, such as fixed-deposit or loan accounts, can implement the contract.
- **Flexibility**: an account class can also implement other interfaces, such as loan or credit services.

Note 51

Multiple Inheritance Through Interfaces

- **Java restriction**: a class cannot inherit from multiple classes, helping avoid conflicts and ambiguity.
- **Interface approach**: one class can implement multiple interfaces and combine contracts from different sources.
- **Banking example**:
- **`BankServices`**: defines `deposit()`.
- **`LoanServices`**: defines `applyLoan()`.
- **`Customer`**: implements both interfaces, enabling deposits and loan applications.
- **`BankingSystem`**: creates a `Customer` object and calls both operations.

Note 52

Interface Categories in Java

- **Normal interface**: standard interface that may contain multiple abstract methods, plus permitted default and static methods; examples include `List` and `Map`.
- **Nested interface**: declared inside a class or another interface, often for encapsulation; example: `Map.Entry`.
- **Marker interface**: empty interface used to tag a class with a capability or property; examples: `Serializable` and `Cloneable`.
- **Functional interface**: has exactly one abstract method, also called a Single Abstract Method (SAM) interface; examples: `Runnable` and `Callable`.

Note 53

Functional Interfaces and Lambda Expressions

- **Functional interface**: interface with exactly one abstract method; it may also contain multiple default or static methods.
- **Lambda expressions**:
- **Purpose**: provide a concise implementation of the single abstract method, supported from Java 8.
- **Runnable example**: `() -> System.out.println("New thread created")` supplies the `run()` behavior without a separate implementation class.
- **Payment example**: a `PaymentProcessor` lambda implements `processPayment(double amount)`; calling `processPayment(250.75)` processes $250.75.
- **`@FunctionalInterface` annotation**:
- **Role**: asks the compiler to enforce the single-abstract-method rule; it is optional.

- **PaymentProcessor example**: its sole method accepts an amount and returns `true` for success or `false` for failure.
- **Built-in examples**:
- **`Runnable`**: defines `run()` for thread tasks.
- **`Callable`**: defines `call()` and can return a result.
- **`Comparable`**: defines `compareTo()` for object comparison.
- **`ActionListener`**: defines `actionPerformed()` for UI events.
- **Additional rules**:
- **Parent methods**: inherited methods corresponding to `Object` methods do not violate the functional-interface rule.
- **Standard library**: `java.util.function` provides predefined interfaces for lambdas and functional programming.

Note 54

Marker Interfaces in Java

- **Marker interface**: empty interface with no methods or fields, used to tag a class with a capability or property.
- **Purpose**:
- **Runtime or compiler recognition**: signals that special processing may apply.
- **Framework checks**: lets APIs determine whether a class has a particular property.
- **Legacy use**: widely used before annotations and still present in existing systems.
- **Examples**:
- **`Serializable`**: marks objects that can be converted into a byte stream and saved to a file.
- **`Cloneable`**: another standard marker-interface example.

Note 55

Exception Handling and Program Flow

- **Exception**: unexpected runtime situation that interrupts the normal flow of a program.
- **Common causes and examples**:
- **Invalid array index**: can cause `ArrayIndexOutOfBoundsException`.
- **Division by zero**: can cause `ArithmeticException`.
- **Unavailable file**: may cause `FileNotFoundException`.
- **Other causes**: invalid input, network loss, resource limits, null references, type mismatches, database failures, and code errors.
- **Handling tools**:
- **`try` block**: contains code that may cause an exception.
- **`catch` block**: handles a matching exception, such as an invalid array index, allowing execution to continue.
- **`finally` block**: runs whether an exception occurs or not; useful for cleanup.
- **Throwing and custom exceptions**: allow exceptions to be raised deliberately and user-defined exception types to be created.
- **Unhandled exception**: stops the program; code after the error does not execute.
- **Common exception classes**: include `ClassNotFoundException`, `IOException`, `SQLException`, and `RemoteException`.

Note 56

Java Throwable Hierarchy and Errors

- `**Throwable**`: superclass of Java exception and error types.
- **Main branches**:
- `**Exception**`: conditions programs may handle; examples include `NullPointerException`.
- `**Error**`: serious runtime or system problems, such as `StackOverflowError`.
- **Error examples**:
- **Resource or runtime failures**: JVM out of memory, memory leaks, and stack overflow.
- **Other severe issues**: library incompatibility and infinite recursion.
- **Typical response**: errors are generally beyond the developer's control and should not be handled as ordinary exceptions.

Note 57

Types of Java Exceptions

- **Exception categories**:
- **Built-in exceptions**: predefined Java classes for common runtime problems.
- **User-defined exceptions**: custom exception classes created by developers.
- **Built-in exception subtypes**:
- **Checked exceptions**: checked by the compiler during compilation; programs must account for them.
- **Unchecked exceptions**: not checked by the compiler at compile time.

Note 58

Java Exception Types and Custom Exceptions

- **Checked exceptions**: Checked by the compiler; examples include `ClassNotFoundException`, `IOException`, `SQLException`, and `FileNotFoundException`.
- **Unchecked exceptions**: Not checked at compile time; an unhandled unchecked exception does not cause a compilation error.
- **Examples**:
- *Example:* `ArithmeticException` for illegal arithmetic, such as division by zero.
- *Example:* `ClassCastException` when casting to an incompatible class.
- *Example:* `NullPointerException` when accessing a null object.
- *Example:* `ArrayIndexOutOfBoundsException` when using an invalid array index.
- **Custom exception**: Application-specific exception created by defining a class that extends `Exception` or `RuntimeException`.
- **Creation and use**:
- Define a constructor that passes a custom message to the parent class.
- Throw the exception with `throw` and handle it with `try-catch`.
- **Benefits**: Represents business or application errors clearly and supports easier debugging.

Note 59

Java Exception Handling Blocks and Keywords

- **`try-catch`**: Put potentially exception-throwing code in `try`; use `catch` to handle an exception.
- **Multiple exceptions**: Use multiple `catch` blocks, each for a different exception type.
- *Example:* Handle `ArithmeticException`, `ArrayIndexOutOfBoundsException`, and `NumberFormatException` separately.
- **`finally`**: Runs after the `try-catch` sequence, whether an exception occurs or not; commonly used to close files or database connections.
- **Exception-handling keywords**:
- **`try`**: Contains code that may cause an exception.
- **`catch`**: Handles an exception.
- **`throw`**: Manually throws an exception.
- **`throws`**: Declares exceptions a method may throw.
- **`finally`**: Runs code after `try`, regardless of whether an exception occurs.
- **Benefits**: Supports program completion, separates application logic from error handling, propagates errors, provides meaningful messages, and distinguishes error types.

Note 60

JDBC Purpose and Architecture

- **JDBC**: Java API for connecting applications to relational databases, running queries, and managing data; supports databases such as MySQL, Oracle, and PostgreSQL.
- **Architecture components**:
- **Application**: Java program that interacts with a database.
- **JDBC API**: Provides query and result-handling interfaces, including `Driver`, `ResultSet`, `RowSet`, `PreparedStatement`, and `Connection`; includes classes such as `DriverManager`, `Types`, `Blob`, and `Clob`.
- **DriverManager**: Uses suitable drivers to manage database connections.
- **JDBC driver**: Translates communication between the application and database.
- **Access models**:
- **Two-tier**: Java application connects directly to the database through a JDBC driver.
- **Three-tier**: Application communicates through a middle layer, such as an application server, which connects to the database.

Note 61

JDBC Components and Driver Types

- **JDBC components**:
- **JDBC API**: Provides database communication methods and interfaces.
- `java.sql` supplies core connection, query, and result classes and interfaces.
- `javax.sql` extends `java.sql` with features such as connection pooling and data-source management.
- **DriverManager**: Loads the appropriate driver and manages connections.
- **JDBC Test Suite**: Tests driver operations such as insertion, deletion, and updating.
- **JDBC drivers**: Client-side adapters that translate Java requests into database-understood protocols.

- **Driver types**:
1. **Type 1**: JDBC–ODBC Bridge Driver.
1. **Type 2**: Native-API Driver; partially Java.
1. **Type 3**: Network Protocol Driver; fully Java.
1. **Type 4**: Thin Driver; fully Java and commonly used by modern applications. The source describes it as deprecated since Java 8.

Note 62

JDBC Classes and Interfaces

- **Connection management**:
- **`DriverManager`**: Manages drivers and establishes connections.
- **`Connection`**: Represents a session with a database.
- **Statement execution**:
- **`Statement`**: Executes static SQL queries.
- **`PreparedStatement`**: Represents a precompiled SQL statement with parameters for dynamic queries.
- **`CallableStatement`**: Executes database stored procedures.
- **Results and errors**:
- **`ResultSet`**: Represents query results and allows navigation through rows.
- **`SQLException`**: Represents SQL-related exceptions.

Note 63

Connecting to MySQL with JDBC

- **Connection workflow**:
1. **Load driver**: Use `Class.forName("com.mysql.cj.jdbc.Driver")`.
1. **Establish connection**: Call `DriverManager.getConnection(url, username, password)` with a URL such as `jdbc:mysql://localhost:3306/database_name`.
1. **Create statement**: Call `connection.createStatement()`.
1. **Execute update**: Run SQL with `executeUpdate()`; for example, insert student `101`, `Kodewala` into `students` and capture the affected-row count.
1. **Close resources**: Close the `Statement` and `Connection`.
- **Sample application setup**:
- Install MySQL Server and connect to it from the command line.
- Create and select a database, then create and inspect a table.
- The example uses the `demo` database and `students` table.
- **Application structure**: Import `java.sql.*`, define connection details and SQL, load the driver, connect, create a statement, execute the insert, and close resources.
- **Exception handling**: Catch `ClassNotFoundException` if the driver is missing and `SQLException` for connection or query errors.
- **Driver dependency**: Download MySQL Connector/J and add its JAR as an external library.

Note 64

Collection Framework Purpose and Benefits

- **Java collection**: Group of individual objects represented as a single unit.
- **Collection Framework**: JDK 1.2 framework of collection classes and interfaces in `java.util`; its two main root interfaces are `Collection` and `Map`.
- **Framework**: Ready-made architecture of related classes and interfaces for similar tasks.
- **Why it was introduced**: Before JDK 1.2, arrays, `Vector`, and `Hashtable` lacked a shared interface and consistent methods; some `Vector` methods were final, limiting extension.
- **Advantages**:
- **Consistent API**: Interfaces such as `Collection`, `Set`, `List`, and `Map` provide shared methods across implementations.
- **Less programming effort**: Developers can use existing collections rather than designing them from scratch.
- **Performance and quality**: Provides efficient data structures and algorithms, allowing suitable implementations to be selected.

Note 65

Collection Framework Core Interfaces

- **`Iterable`**: Root interface that provides iteration over collection elements.
- Its `iterator()` method returns an iterator.
- **`Collection`**: Extends `Iterable` and provides common operations such as adding, removing, and clearing elements.
- Acts as the foundation for collection classes and interfaces.
- **`Map`**: One of the framework's two main root interfaces, alongside `Collection`.
- **`Collection` versus `Collections`**: `Collection` is an interface; `Collections` is a utility class.
- **`List`**: Child interface of `Collection` for ordered elements; duplicates are allowed.
- Implementations include `ArrayList`, `LinkedList`, `Vector`, and `Stack`.
- *Example:* A classroom registration list can preserve student order and contain students with identical names.
- Lists can be declared using implementations such as `List<T> list = new ArrayList<>();` or `new LinkedList<>()`.

Note 66

ArrayList Structure and Performance

- **`ArrayList`**: Resizable array implementation of `List`; preserves order, allows duplicates, and supports random access.
- Stores objects; primitive values require wrapper types such as `Integer` or `Character`.
- **Internal storage**: Uses an `Object[]` dynamic array.
- A no-argument instance is described as having default capacity 10; initial capacity can be specified, such as `new ArrayList<>(20)`.
- **Resizing**: When full, creates a larger array and copies elements; the described growth formula is `newCapacity = oldCapacity + oldCapacity / 2`.
- **Operation costs**:
- **Index access**: Fast, constant time `O(1)`.
- **Insert or remove away from the end**: Later elements shift, taking linear time `O(n)`.
- **Best use**: Choose for fast random access, frequent additions or removals at the end,

ordered elements, and duplicates; avoid frequent changes near the beginning or middle.

Note 67

LinkedList Structure and Performance

- **`LinkedList`**: `List` implementation that stores elements as linked nodes rather than in a continuous array; useful for frequent additions and removals.
- **Doubly linked structure**: Each node contains data, a link to the next node, and a link to the previous node.
- **Dynamic size**: Grows or shrinks as elements are added or removed; it has no fixed capacity.
- **Access cost**: Has no direct index-based storage; reaching an indexed element requires traversal from the head or tail, taking `O(n)` time.
- **Contrast with `ArrayList`**: Linked nodes support a different storage structure, while `ArrayList` uses an array and offers constant-time indexed access.

Note 68

Choosing Between LinkedList and ArrayList

- **LinkedList**: Best when insertions or deletions occur frequently, especially at the beginning or end.
- **Why**: Updates links instead of shifting elements.
- **Access cost**: Finding an element by index requires traversal, taking `O(n)`.
- **Insertion cost**:
- **At the head or tail**: `O(1)`.
- **At a specified middle index**: `O(n)` overall; traversal takes `O(n)`, then changing links takes `O(1)`.
- *Example:* Random insertion is not truly `O(1)` because the target position must first be reached.
- **ArrayList**: Better for frequent index-based access, which takes `O(1)`, and for insertions or removals mostly at the end.
- **LinkedList structure**: Elements are nodes connected by links rather than stored in continuous memory; this avoids shifting during insertion and removal.

Note 69

Set Interface and Set Implementations

- **Set**: Collection that prevents duplicates; it is typically unordered.
- **Use**: Store unique objects without duplication.
- **Implementations**: `HashSet`, `LinkedHashSet`, and `TreeSet`.
- **HashSet**: Uses hashing; does not preserve insertion order and allows one `null` element.
- **Storage**: Elements are placed in buckets based on their hash codes.
- *Example:* A repeated `Smartphone` is ignored; iteration order is not guaranteed.
- **LinkedHashSet**: Combines hashing with a doubly linked list to preserve insertion order.
- **Storage**: Backed by `LinkedHashMap`; elements are map keys with a dummy value.

- *Example*: A repeated `Smartphone` is ignored, while insertion order is retained.
- **TreeSet**: Stores unique elements in ascending sorted order using a self-balancing Red-Black tree.
- **Storage**: Backed by `TreeMap`; ordering uses natural ordering or a `Comparator`.
- **Restrictions**: Does not allow `null`; is not thread-safe.
- **Performance**: `add()`, `remove()`, and `contains()` take `O(log n)`.

Note 70

Hashing, Equality, and HashSet Internals

- **`hashCode()`**: Produces an integer used to help determine an object's bucket; different objects may share a hash code.
- **`equals()`**: Checks whether objects in the same bucket are equal and therefore duplicates.
- **Contract**:
- **Equal objects**: Must have the same hash code.
- **Different hash codes**: Objects are treated as unequal by hash-based collections.
- **HashSet storage**: Backed by a `HashMap`; each element is stored as a key with a constant dummy value, such as `PRESENT`.
- **Adding an element**:

1. **Calculate hash**: Compute the element's hash code and process it to determine a bucket.
1. **Check collisions**: If the bucket contains entries, use `equals()` to check for duplicates.
1. **Store entry**: Add non-equal colliding elements to a linked list or, in Java 8+, a balanced tree when applicable.

- **Bucket index**: Commonly calculated using `(n - 1) & hash`, where `n` is the bucket-array capacity.
- **Capacity and resizing**:
- **Defaults**: Capacity `16`; load factor `0.75`.
- **Rehashing**: When entries exceed `capacity × load factor`, capacity doubles and entries are redistributed.

Note 71

TreeSet Equality and Ordering Rules

- **Duplicate detection**: `TreeSet` uses `compareTo()` or `compare()`, rather than `hashCode()` and `equals()`.
- **Comparison result zero**: Elements are treated as duplicates, even if `equals()` says they differ.
- **Null handling**: Adding `null` throws `NullPointerException`.
- **Thread safety**: `TreeSet` is not thread-safe; `Collections.synchronizedSortedSet()` can provide a synchronized wrapper.

Note 72

Iterating Over Lists and Sets

- **For-each loop**: Readable, simple way to visit every collection element; internally uses an

`Iterator`.
- **Best for**: Reading elements without modifying the collection during iteration.
- **Iterator**: Interface for traversing collections, offering more control than a for-each loop.

Note 73

Map Interface and Common Implementations

- **Map**: Stores key-value pairs; values are accessed using keys.
- **Keys**: Must be unique; each key identifies one value.
- **Values**: May repeat across different keys.
- *Example:* Look up a student's name using their ID.
- **HashMap**: Uses hashing; does not guarantee ordering.
- **Lookup**: A value is retrieved using its key.
- **Hashing**: Converts data, such as a string, into a hash code used to locate a bucket.
- **LinkedHashMap**: Maintains key-value pairs in insertion order using a doubly linked list.
- **TreeMap**: Stores entries in key-sorted order using a Red-Black tree.

Note 74

How HashMap Stores and Retrieves Entries

- **Bucket selection**: `HashMap` computes the key's `hashCode()` and derives a bucket index, commonly with `hash & (n - 1)`.
- **Insertion with `put()`**:
1. **Find bucket**: Calculate the key's hash and bucket index.
1. **Check bucket**: Add the entry if the bucket is empty; otherwise compare keys using `equals()`.
1. **Resolve collision**: Store different keys sharing a bucket together; an equal key replaces its value.
- **Collision structures**:
- **Earlier Java versions**: Entries in a bucket were stored in a linked list.
- **Java 8+**: A heavily populated bucket can be converted into a Red-Black tree; the notes give more than eight entries as the threshold.
- **Retrieval with `get()`**: Recomputes the bucket from `hashCode()`, then uses `equals()` to locate the exact key.
- *Example:* `map.put("Apple", "iPhone11")` stores a value under the key `"Apple"`.

Note 75

ConcurrentHashMap and Collection Utilities

- **ConcurrentHashMap**: Thread-safe map designed for concurrent access without locking the entire map.
- **Concurrency**: Supports safe reads and writes by multiple threads with limited blocking.
- **Nulls**: Does not allow `null` keys or values.
- **Use**: Suitable for high-performance, multi-threaded tasks such as caches and counters.
- **Collections**: Utility class containing static methods that operate on or return collections.

- **Distinction**: `Collection` is a root interface; `Collections` is a helper class.
- **Synchronized wrappers**: Provide synchronized versions of collections.
- **Methods**: `synchronizedList()`, `synchronizedSet()`, and `synchronizedMap()`.
- **Unmodifiable views**: Prevent modifications through the returned view.
- **Methods**: `unmodifiableList()`, `unmodifiableSet()`, and `unmodifiableMap()`.

Note 76

Java Stream API Fundamentals

- **Stream API**: Java 8 feature for processing collections in a declarative, functional style.
- **Operations**: Filter, transform, and collect data with less repetitive loop and conditional code.
- **Java Stream**: Processes collection objects while handling iteration internally.
- **Not the same as**: `InputStream` and `OutputStream`, which handle byte streams.
- **Processing pipeline**:

1. **First operation**: Processes an element and passes it to the next operation.
1. **Chained operations**: Each operation can modify an element or leave it unchanged.
1. **Element flow**: Elements pass through the connected operations in sequence.

Note 77

Java Stream Creation and Core Features

- **Creation from a collection**: Call `stream()` on a collection such as a `List`.
- *Example:* A list containing `"Kodewala"`, `"Academy"`, and `"Bangalore"` can be processed with `forEach(item -> System.out.println(item))`.
- **Purpose**: Processes elements one by one in a functional style; it is not a data structure and does not store data.
- **Features**:
- **Functional style**: Uses lambdas to express processing logic.
- **Lazy evaluation**: Intermediate operations wait until a terminal operation is called.
- **Chaining**: Combines operations such as `filter`, `map`, and `collect` into a pipeline.
- **Parallel processing**: Supports parallel execution through `parallelStream()`.

Note 78

Sequential and Parallel Streams

- **Sequential stream**: Processes data in a single thread; this is the default stream mode.
- **Parallel stream**: Splits work across multiple threads, potentially improving performance on large data sets.
- **Common-pool parallelism**: The material gives the typical estimate `CPU cores − 1` threads.
- *Example:* An 8-core system typically uses 7 threads.
- **Checking parallelism**: `ForkJoinPool.commonPool().getParallelism()` prints the common pool's parallelism level.

Note 79

Intermediate Stream Operations and Lazy Evaluation

- **Intermediate operation**: Returns another stream and builds a processing pipeline without immediately processing elements.
- **Purpose**:
- **Transformation and filtering**: Changes elements or selects those matching conditions.
- **Chaining**: Multiple intermediate operations can be combined before a terminal operation.
- **Efficiency**: Laziness supports large or infinite sources and avoids unnecessary work.
- **Pipeline execution**:

1. **Build**: Declare intermediate operations; no data processing occurs yet.
1. **Trigger**: Call a terminal operation, such as `forEach()`.
1. **Process**: Each element passes through the pipeline; for example, `filter()` runs before `map()`, then the result reaches `forEach()`.

- **Fused processing**: Multiple intermediate operations can run in a single pass over the data.
- **Short-circuiting**: Some terminal operations can stop early when the result is determined, such as `findFirst()` or `anyMatch()`.

Note 80

Common Intermediate Stream Operations

- **`filter()`**: Keeps elements that satisfy a `Predicate`; removes elements for which the condition is false.
- **`map()`**: Transforms each element using a supplied function and produces transformed values.
- **`flatMap()`**: Converts each element into zero or more elements, then flattens them into one stream.
- **Use**: Handles nested structures, such as lists inside lists or strings containing words.
- **`distinct()`**: Removes duplicates, leaving unique elements.
- **`limit(n)`**: Keeps only the first `n` elements.
- **`peek()`**: Observes elements without changing them; useful for debugging or logging, and runs only when a terminal operation triggers the pipeline.
- **`sorted()`**: Produces a sorted stream without changing the original list.
- **Default order**: Natural order, such as alphabetical or ascending numeric order.
- **Custom order**: Supply a comparator, for example, sorting strings by length.
- **Primitive mappings**: `mapToInt()`, `mapToDouble()`, and `mapToLong()` convert elements to primitive streams.
- **`skip(n)`**: Discards the first `n` elements; returns an empty stream if fewer than `n` elements exist.
- **Common use**: Combine with `limit()` to paginate or slice data.

Note 81

Terminal Stream Operations and Collectors

- **Terminal operation**: Triggers stream processing, returns a final non-stream result, and consumes the stream.

- **Reuse restriction**: A stream can have only one terminal operation; after execution, it cannot be reused or further chained.
- **Common terminal operations**:
- **`forEach()`**: Performs an action for each element; returns nothing.
- **`toArray()`**: Converts stream elements into an array, optionally using an array generator.
- **`reduce()`**: Combines elements into one value using an identity and accumulator.
- *Example:* Reducing `[1, 2, 3, 4]` with identity `0` and addition gives `10`.
- **`collect()`**: Accumulates elements into a collection, map, or other container using a `Collector`.
- **`min()` and `max()`**: Find the smallest or largest element according to a comparator.
- **`count()`**: Returns the number of elements as a `long`.
- **Collector methods**:
- **`toList()`**: Collects elements into a list.
- **`toSet()`**: Collects elements into a set.
- **`toCollection()`**: Collects elements into a specified collection type.
- **`joining()`**: Concatenates strings.
- **`counting()`**: Counts elements.
- **`groupingBy()`**: Groups elements using a classifier function.
- **`partitioningBy()`**: Splits elements according to a predicate.

Note 82

Stream Practice Problems and Supporting Methods

- **Practice problems**:
- **Numbers and lists**: Find the second-highest number, duplicates, the longest string, or the sum of squares of even numbers.
- **Strings and characters**: Count character frequencies, reverse each word, or find the first non-repeated character.
- **Grouping and combining**: Group strings by length; merge two lists and remove duplicates; sort strings by length.
- **Selection and retrieval methods**:
- **`sorted(Comparator.reverseOrder())`**: Sorts in descending order.
- **`skip(n)`**: Skips the first `n` elements; useful with sorting to select ranked values.
- **`findFirst()`**: Returns the first matching stream element.
- **`orElse()`**: Supplies a default value when a result is empty.
- **`orElseThrow()`**: Throws an exception when a result is empty.
- **Character and collection methods**:
- **`chars()` and `mapToObj()`**: Convert string character values from an `IntStream` into object elements.
- **`Collections.frequency()`**: Counts occurrences in a collection.
- **`StringBuilder.reverse()`**: Reverses characters.
- **`indexOf()` and `lastIndexOf()`**: Find the first or last index of a character or string.
- **Other useful methods**:
- **`Stream.concat()`**: Concatenates two streams.
- **`Comparator.comparingInt()`**: Compares elements using an integer property.
- **`mapToInt()` and `sum()`**: Convert to an `IntStream` and sum its values.
- **`Collectors.toSet()`, `toList()`, `joining()`, `groupingBy()`, and `counting()`**: Collect, join, group, or count results.

Note 83

Maven and the pom.xml Configuration File

- **Maven**: Build automation and project management tool for Java and other JVM languages.
- **Main uses**:
- **Dependency management**: Downloads and uses external libraries.
- **Build automation**: Compiles code, runs tests, and creates JAR or WAR files.
- **Project organization**: Uses a standard folder structure.
- **`pom.xml`**: XML-based Project Object Model file that configures a Maven project.
- **Project information**: Name, version, and packaging type such as JAR or WAR.
- **Dependencies**: Libraries required by the project.
- **Build settings**: Plugins and compiler settings.
- **Repositories**: Locations from which Maven retrieves dependencies.
- **Key elements**:
- **`groupId`**: Unique identifier for the project's organization.
- **`artifactId`**: Project name or artifact identifier.
- **`version`**: Project version.
- **`packaging`**: Output type, such as `jar` or `war`.
- **`dependencies`**: Required libraries.
- **`plugins`**: Tools that add build features.

Note 84

Maven Repositories and Commands

- **Repository**: Location where Maven stores or retrieves JAR files.
- **Repository types**:
- **Local**: `~/.m2/repository`; caches downloaded dependencies on the machine.
- **Central**: Maven's default public repository at `https://repo.maven.apache.org/maven2`.
- **Remote**: Custom repository, often used by companies for internal libraries.
- **Dependency lookup flow**:

1. **Check local repository**: Reuse a dependency if it is already cached.
1. **Check remote sources**: Retrieve it from a configured company or remote repository when applicable.
1. **Check Maven Central**: Download from Central if needed; save the result locally.

- **Common commands**:
- **`mvn clean`**: Removes previous build output, including the `target/` directory.
- **`mvn compile`**: Compiles source code.
- **`mvn test`**: Runs tests.
- **`mvn package`**: Builds a JAR or WAR file.
- **`mvn install`**: Installs the package in the local Maven repository.
- **`mvn deploy`**: Deploys the package to a remote repository.

Note 85

Installing Maven on Windows

- **Download**: Get the appropriate binary package from Maven's official website; for Windows,

the material specifies the binary ZIP archive.
- **Extract and place files**:
1. **Extract**: Unzip the downloaded Maven archive.
1. **Move**: Copy the extracted folder into `C:\Program Files`.
1. **Record location**: Copy the installation path, such as `C:\Program Files\apache-maven-3.9.9`.
- **Set environment variables**:
1. **Create `MAVEN_HOME`**: Set its value to the Maven installation folder.
1. **Edit `Path`**: Add Maven's `bin` directory, such as `C:\Program Files\apache-maven-3.9.9\bin`.
- **Verify installation**: Open a new Command Prompt and run `mvn -version`; a version message confirms setup.

Note 86

Creating and Running a Maven Java Project

- **Prerequisite**: Internet access is needed to generate the sample project and retrieve dependencies.
- **Generate a standalone project**: Run the archetype command with project coordinates and non-interactive mode.
- `mvn archetype:generate -DgroupId=com.example -DartifactId=helloworld -DarchetypeArtifactId=maven-archetype-quickstart -DinteractiveMode=false`
- **Compile**: Change to the `helloworld` project directory and run `mvn compile`.
- **Run**: Execute the sample application's main class with `mvn exec:java -Dexec.mainClass="com.example.App"`.

Note 87

SDLC Phases and the Role of JUnit

- **SDLC**: Step-by-step process for planning, developing, testing, and deploying software; aims for quality, cost-effectiveness, and timely delivery.
- **Phases**:
1. **Requirement analysis**: Gather and document client needs.
- *Example:* A calculator must support addition, subtraction, multiplication, and division.
1. **Design**: Plan architecture, database, interface, classes, methods, and data flow.
1. **Implementation**: Write code according to the design.
- *Example:* Create `Calculator.java` with `add()` and `divide()` methods.
1. **Testing**: Verify that code behaves as expected; JUnit supports unit testing here.
- *Example:* Check that `add(2, 3)` returns `5`.
1. **Deployment**: Release software to production, such as a server or app store.
1. **Maintenance**: Fix bugs and add updates; rerun tests for regression testing.
- **JUnit's role**: Tests small units, such as individual methods or classes, in isolation during the testing phase.

Note 88

JUnit 4 Features and Benefits

- **JUnit**: Java framework for testing small units of code, such as methods and classes.
- **JUnit 4**: Annotation-based framework that integrates readily with Maven and Gradle.
- **Testing purpose**:
- **Verify correctness**: Check individual methods and classes.
- **Find bugs early**: Detect problems during development, before production.
- **Benefits**:
- **Automated testing**: Runs checks without manually inspecting outputs.
- **Faster feedback**: Runs many tests quickly.
- **Code quality**: Checks new and existing functionality.
- **Regression testing**: Reruns old tests after changes to detect breakage.
- **Readable tests**: Annotations such as `@Test`, `@Before`, and `@After` clarify test structure.
- **Tool integration**: Works with Maven, Gradle, Jenkins, and GitHub Actions in CI/CD pipelines.

Note 89

Setting Up a JUnit 4 Calculator Test

- **Add dependency**: Include the JUnit 4 dependency in `pom.xml`; Maven downloads it after the file is saved.
- **Create the test package**: Under `src/test/java`, create the package `com.kodewala`.
- **Create the test class**: Add `CalculatorTest.java` inside that package to test `Calculator.java`.

Note 90

Running JUnit Tests and Interpreting Code Coverage

- **Run a test in Eclipse**: Right-click `CalculatorTest.java` and select **Run As** → **JUnit Test**.
- **Green bar**: All tests passed.
- **Red bar**: One or more tests failed; error details are shown.
- **Code coverage**: Measures how much source code executes during automated tests.
- **Purpose**: Reveals tested and untested code, helps identify dead code, and supports reliability.
- **Target**: Around 80% or higher is generally considered good for many projects.
- **Limitations**: 100% may be impractical, especially for error-handling or logging code.
- **Best practice**: Prioritize meaningful tests over maximizing the percentage.

Note 91

Java Threads and Their Lifecycle

- **Thread**: Lightweight unit of execution within a process; Java programs start with at least the `main` thread.
- **Main thread**: The JVM creates it to execute the `main()` method.
- **Additional threads**: Developers create them from `main()` or other methods; print

`Thread.currentThread().getName()` to check the current thread.
- *Example:* A restaurant is a process, and its waiters are threads serving different tables.
- **Thread creation methods**:
- **Extend `Thread`**: Define a thread by subclassing `Thread`.
- **Implement `Runnable`**: Define a task separately from the thread; preferred for flexibility because the class can extend another class.
- **Use `ExecutorService`**: Preferred in real applications for thread pooling and lifecycle management.
- **Lifecycle states**:
- **New**: Created but not started, such as `new Thread()`.
- **Runnable**: Started and eligible for CPU time.
- **Running**: Actively executing `run()`; often described separately from Java's formal `RUNNABLE` state.
- **Blocked**: Waiting to acquire a monitor lock.
- **Waiting / Timed waiting**: Paused indefinitely or for a specified time.
- **Terminated**: `run()` has completed.
- **State transitions and thread methods**:
- **Waiting to running**: A thread returns through `RUNNABLE` before it can execute again.
- **`wait()` vs `sleep()`**: `wait()` releases the object's lock; `sleep()` pauses for a time without releasing locks.
- **`join()`**: Makes the calling thread wait until another thread completes.
- **Direct `run()` call**: Executes like a normal method in the current thread; it does not create concurrent execution.
- **Restarting a completed thread**: Not allowed; calling `start()` again throws `IllegalThreadStateException`.

Note 92

Synchronization, Locks, and Concurrency Utilities

- **Synchronization**: Controls access to shared resources to prevent race conditions and inconsistent data.
- *Example:* Concurrent increments may read the same counter value and produce a final count lower than expected.
- **`synchronized`**: Provides mutual exclusion so only one thread at a time executes protected code.
- **Synchronized method**: Locks the relevant object or class for the method.
- **Synchronized block**: Locks a specified object around only the critical section; prefer it when a smaller lock scope is suitable.
- **Monitor behavior**: Each object and class has an intrinsic lock; a thread unable to acquire it becomes `BLOCKED` until release.
- **`ReentrantLock`**: Explicit lock with additional control beyond `synchronized`.
- **`tryLock()`**: Attempts to acquire the lock without waiting if unavailable.
- **Timed `tryLock()`**: Waits for a specified duration, then gives up if the lock remains unavailable.
- **Other advantages**: Supports features such as interruptible locking and explicit lock management; ensure locks are released appropriately.
- **`ReadWriteLock`**: Allows concurrent readers but exclusive writing.
- **Implementation**: `ReentrantReadWriteLock` is a common implementation.
- *Example:* Many users can read bank balances simultaneously, while deposits or withdrawals require exclusive access.

- **`Semaphore`**: Limits simultaneous access to a resource using a fixed number of permits.
- **Fair mode**: `new Semaphore(initialCount, true)` requests fair, FIFO access.
- *Example:* Restrict concurrent use of database connections or a limited-size resource pool.
- **Choosing a mechanism**:
- **`synchronized`**: Simple mutual exclusion with automatic lock release.
- **`ReentrantLock`**: Use when timed, interruptible, or other explicit lock control is needed.
- **`ReadWriteLock`**: Use for workloads with many reads and few writes.
- **`Semaphore`**: Use when a resource has a limited number of concurrent access slots.

Note 93

Inter-Thread Communication and Producer–Consumer Coordination

- **Inter-thread communication**: Coordinates threads that share resources, commonly through `wait()`, `notify()`, and `notifyAll()`.
- **`wait()`**: Releases the object's lock and pauses the current thread until notification or timeout.
- **`notify()`**: Wakes one thread waiting on the same object.
- **`notifyAll()`**: Wakes all threads waiting on the same object.
- **Method location**: These methods belong to `Object`, not `Thread`.
- **Synchronization requirement**: Call `wait()`, `notify()`, and `notifyAll()` while holding the object's monitor, typically inside a synchronized block; otherwise `IllegalMonitorStateException` can occur.
- **Producer–consumer pattern**:
- **Producer**: Waits when previously produced data has not yet been consumed.
- **Consumer**: Waits when no new data is available.
- **Shared object**: Coordinates the producer and consumer's access to shared state.
- **Blocking-queue alternative**: Use `BlockingQueue` methods to coordinate without explicit `wait()` and `notify()`.
- **`put()`**: Waits when the queue is full.
- **`take()`**: Waits when the queue is empty.

Note 94

Executor Framework, Thread Pools, and Asynchronous Results

- **Executor Framework**: Java 5 API in `java.util.concurrent` that separates task submission from thread management and supports thread pools.
- **Core interfaces and classes**:
- **`Executor`**: Basic task execution interface with `execute(Runnable)`.
- **`ExecutorService`**: Adds task submission and lifecycle controls, including `submit()` and `shutdown()`.
- **`Executors`**: Utility class with factory methods for thread pools.
- **`Callable<V>`**: Task that returns a result and may throw checked exceptions; uses `call()`.
- **`Future<V>`**: Represents the result of an asynchronous task.
- **Thread-pool types**:
- **Fixed pool**: `newFixedThreadPool(n)` uses a fixed number of threads for stable concurrency.
- **Cached pool**: `newCachedThreadPool()` reuses idle threads and can create an unbounded

number; suited to many short-lived tasks.
- **Single-thread executor**: `newSingleThreadExecutor()` runs tasks sequentially on one thread.
- **Scheduled pool**: `newScheduledThreadPool(n)` runs tasks after delays or periodically.
- **`Runnable` vs `Callable`**:
- **`Runnable`**: Uses `run()`, returns no result, and cannot declare checked exceptions.
- **`Callable`**: Uses `call()`, returns a result, and can throw checked exceptions; commonly submitted with `ExecutorService`.
- **`Future` operations**: Check whether a task is done, cancel it, or retrieve its result when ready.
- **Why use pools**: Reusing a bounded set of threads avoids the overhead and resource cost of creating a new thread for every task.
- *Example:* Use `Executors.newFixedThreadPool()` for thousands of short-lived tasks.

Note 95

Common Multithreading Problems and Solutions

- **Race condition**: Concurrent access or updates produce results dependent on thread timing.
- **Solutions**: Protect updates with synchronization or locks, or use atomic classes such as `AtomicInteger`.
- **Debugging**: Use logging and thread-analysis tools to inspect interleavings and shared-state access.
- **Deadlock**: Threads wait indefinitely for locks held by one another.
- **Detection**: Inspect thread dumps or monitoring tools for threads waiting indefinitely.
- **Prevention**: Acquire locks in a consistent global order, avoid nested locks where possible, or use timed `tryLock()`.
- **Waiting for another thread**: Use `join()` when one thread must finish before the caller proceeds.
- **More complex coordination**: Use `wait()` and `notify()` on a shared monitor inside synchronized sections.
- **Producer–consumer coordination**: A blocking queue's `put()` and `take()` handle waiting for capacity or data.
- **Lock choice**: Use `synchronized` for straightforward mutual exclusion; choose `ReentrantLock` for advanced locking controls.

Note 96

Spring Framework and Its Modules

- **Spring Framework**: Lightweight, open-source framework for Java applications, especially enterprise applications.
- **Lightweight meaning**: Use only needed modules; run in environments such as Tomcat or Jetty without requiring a full Java EE application server.
- **Why it is popular**: Simplifies Java development and reduces traditional Java EE complexity.
- **Core module groups**:
- **Spring Core, Beans, and Context**: Provide IoC and dependency injection, bean creation and lifecycle, `ApplicationContext`, and resource loading.

- **Spring Web and Web MVC**: Support HTTP handling, REST APIs, and MVC web applications.
- **Spring JDBC and ORM**: Simplify JDBC operations and integrate Hibernate, JPA, and other ORM tools.
- **Spring Data JPA**: Provides CRUD repositories and custom query creation.
- **Spring Security**: Supports authentication, authorization, password hashing, and OAuth2.
- **Spring Test**: Supports unit and integration testing with mock environments.
- **Frequently used add-ons and other modules**:
- **Spring Boot**: Simplifies setup with auto-configuration and embedded servers.
- **Spring Actuator**: Provides production monitoring and health-check endpoints.
- **Spring Cloud**: Supports distributed-system patterns such as configuration servers, service registries, and load balancing.
- **Spring Batch**: Supports robust, scalable batch-processing jobs.
- **Spring Scheduling**: Enables scheduled tasks with `@Scheduled`.

Note 97

Spring IoC Containers and Bean Configuration

- **Inversion of Control (IoC)**: Transfers object creation and dependency management from application code to the Spring framework or container.
- **Container responsibilities**: Creates, initializes, assembles, and manages bean lifecycles.
- **Dependency injection**: The container supplies required objects instead of developers creating and wiring them manually.
- **Bean configuration inputs**: Provide POJO classes and configuration so the IoC container can create and manage beans.
- **Configuration styles**: XML, Java-based configuration, and annotations.
- **Container types**:
- **`BeanFactory`**: Basic IoC container and foundation interface.
- **Loading**: Creates beans lazily when requested.
- **Use**: Lightweight option for resource-constrained or small applications.
- **`ApplicationContext`**: Extends `BeanFactory` with additional features and Spring integration.
- **Loading**: Typically preloads beans when the container starts.
- **Use**: Suited to enterprise applications and supports XML, annotation-based, and Java-based configuration.

Note 98

Spring Bean Configuration Choices

- **XML configuration**: Prefer for legacy projects, clear separation of configuration from code, or teams comfortable with XML.
- **Java-based configuration**: Prefer for type safety, control over bean creation, or programmatic setup of complex and third-party beans.
- **Annotation-based configuration**: Prefer for modern applications, less boilerplate, configuration near classes, and modular projects using component scanning.

Note 99

Spring Beans and Their Lifecycle

- **Spring bean**: Java object managed by the Spring IoC container, which controls its lifecycle and dependencies.
- **Registration**:
- **Component scanning**: Registers classes annotated with `@Component`, `@Service`, `@Repository`, and related annotations.
- **Configuration method**: Registers an object returned by a `@Bean` method in a `@Configuration` class.
- **Lifecycle stages**:

1. **Instantiation**: Spring creates an instance from the bean definition.
1. **Dependency injection**: Spring supplies dependencies through constructors or setters.
1. **Initialization**: Calls setup callbacks such as `@PostConstruct` or `InitializingBean`.
1. **Usage**: The bean participates in application operations and interacts with other beans.
1. **Destruction**: Calls cleanup callbacks such as `@PreDestroy` or `DisposableBean`.

- **Lifecycle callback choice**:
- **`@PostConstruct`**: Clean setup option that requires no interface and works with Spring DI.
- **`afterPropertiesSet()`**: Useful when Spring lifecycle control is needed, especially in framework-level or reusable components.
- **Other options**: `@PreDestroy`, `DisposableBean`, and custom `init-method` or `destroy-method` callbacks.

Note 100

Spring Bean Scopes

- **Bean scope**: Defines a bean's instance count, lifecycle, and visibility within the container.
- **Scopes**:
- **`singleton`**: One shared instance per Spring container; default scope.
- **`prototype`**: Creates a new instance each time the bean is requested.
- **`request`**: One instance per HTTP request; web applications only.
- **`session`**: One instance per user session; web applications only.
- **`application`**: Shared across requests and sessions within a web application context.
- **Selection**: Choose a scope to suit instance needs and manage resources, performance, and memory efficiently.
- **Example**:
- *Example:* A `UserService` can be singleton, while a `ReportGenerator` can use prototype scope for a new instance per request.

Note 101

Dependency Injection Methods

- **Dependency injection (DI)**: Supplying a target object with the dependent objects it needs.
- **Injection methods**:
- **Setter injection**: Supplies dependencies through setter methods.
- **Use**: Optional dependencies or dependencies that may be reconfigured.
- **Trade-off**: Flexible and supports partial injection.
- **Constructor injection**: Supplies dependencies through the constructor.

- **Use**: Mandatory dependencies; common for services and configuration classes.
- **Advantages**: Supports immutability, required fields, and simpler unit testing.
- **Field injection**: Injects directly into fields, often with `@Autowired`.
- **Advantage**: Minimal boilerplate.
- **Disadvantages**: Harder to test and violates encapsulation; generally avoid in clean, testable designs.
- **Field-injection troubleshooting**: Check that component scanning is enabled and the class is managed by Spring when an injected field is `null`.

Note 102

Qualifiers and Spring Profiles

- **`@Qualifier`**: Selects a specific bean when multiple beans of the same type are available for autowiring.
- **Example:** Register `EmailService` and `SMSService` as `emailService` and `smsService`, then use `@Qualifier("emailService")` where needed.
- **Without a qualifier**: Multiple matching beans can cause `NoUniqueBeanDefinitionException`.
- **Spring profiles**: Separate configuration and beans for different environments, such as development, testing, and production.
- **Profile use cases**:
- **Environment-specific resources**: Use H2 for development and Oracle for production.
- **Feature selection**: Enable or disable caching by environment.
- **Activation**: Mark configuration with `@Profile("dev")` or `@Profile("prod")`; activate through properties, command-line arguments, or environment variables.

Note 103

Spring Bean Registration and Troubleshooting

- **`@Component` versus `@Bean`**:
- **`@Component`**: Use for classes you own and register through component scanning.
- **`@Bean`**: Use for third-party classes or when custom object-creation logic is needed.
- **`NoSuchBeanDefinitionException`**: Indicates Spring cannot find a bean matching the injection request.
- **Checks**: Confirm the bean is registered and the component-scan path includes its package.
- **Fix**: Add an appropriate annotation such as `@Component` or `@Service`, or define it with `@Bean`.
- **`BeanFactory` and `ApplicationContext`**:
- **`BeanFactory`**: Core container that supports lazy loading.
- **`ApplicationContext`**: More feature-rich container.

Note 104

Externalizing Spring Configuration Values

- **Property injection**: Use `@Value("${api.key}")` to inject a value from configuration into a Spring bean.
- **Configuration sources**: Store settings in `application.properties` or `application.yml` rather than hardcoding them.
- **Production secrets**: Use an external configuration server or encrypted vault for sensitive values such as API keys.

Note 105

Spring Data Core Concepts and Features

- **Spring Data**: Spring ecosystem project that simplifies data access and persistence with repository-based APIs.
- **Core concepts**:
- **Entity**: Domain object mapped to a database table.
- **Repository**: Interface for data-access operations.
- **`JpaRepository`**: Provides ready-made CRUD, pagination, and sorting operations.
- **Query derivation**: Generates queries from method names such as `findByNameAndAge(...)`.
- **Paging and sorting**: Uses types such as `Page`, `Pageable`, and `Sort`.
- **Benefits**:
- **Less boilerplate**: Avoids repetitive JDBC/JPA data-access code and simplifies the DAO layer.
- **Consistent access**: Supports common patterns across relational and NoSQL stores, including JPA, MongoDB, and Redis.
- **Additional features**: Provides auditing support and custom queries through `@Query`.

Note 106

Spring Data JPA Entities and Repositories

- **Basic workflow**:
1. Define a JPA entity.
1. Create a repository interface.
1. Use the repository in a service or controller.
- **Entity mapping**:
- **`@Entity`**: Marks a class for persistence; by default, its table name matches the class name.
- **`@Table(name = "...")`**: Specifies a different table name.
- **`@Id`**: Marks the primary-key field.
- **`@GeneratedValue`**: Selects how primary-key values are generated.
- **Repository**:
- **`JpaRepository<Entity, IdType>`**: Provides CRUD, pagination, and sorting operations.
- **Example:** `StudentRepository` can extend `JpaRepository<Users, Long>` to access these operations.
- **Identifier strategies**:
- **`IDENTITY`**: Database generates the primary-key value.
- **`AUTO`**: JPA chooses a strategy based on the database.
- **`SEQUENCE`**: Uses a database sequence; often preferred for production when supported.

- `**TABLE**`: Tracks identifiers in a separate table; generally avoid unless necessary.
- **Caution**: `AUTO` behavior varies by database, so it may be unsuitable for large-scale systems.

Note 107

Choosing Derived Queries and Custom Queries

- **Derived query methods**: Let Spring generate queries from repository method names.
- **Use when**: The query is simple and follows naming conventions.
- **Example:** `findByCourse(String course)` can generate `SELECT * FROM student WHERE course = ?`.
- `**@Query**`: Defines a custom JPQL or native SQL query in a repository interface.
- **Use when**: Joining multiple entities, selecting specific fields, applying complex filters, or requiring direct SQL control.
- **Native SQL**: Set `nativeQuery = true` in `@Query` when using SQL directly.

Note 108

Spring Transactions

- `**@Transactional**`: Defines a transactional boundary; begins a transaction at method start, commits on success, and rolls back on exceptions.
- **Placement**:
- **Scope**: Apply at class or method level; method-level settings override class-level settings.
- **Visibility**: The material describes it as applying to public methods.
- **Service-layer practice**:
- **Business operations**: Operations such as registration or placing an order may involve multiple database calls.
- **Atomicity**: A service-layer transaction ensures all operations succeed or fail together.
- **Repository design**: Keeps repository methods reusable and simple.
- **Read-only transaction**: `@Transactional(readOnly = true)` marks a transaction as read-only and may help optimize performance.

Note 109

JPA, Hibernate, and Spring Data JPA

- **JPA**: Persistence specification that defines object-relational mapping and persistence operations.
- *Example:* Map a Java `Users` class to a `users` table with `id` and `name` columns.
- **Hibernate**: JPA implementation that translates persistence operations into database commands and executes them.
- *Example:* On a request to save a `User` object, Hibernate generates and runs the corresponding database command.
- **Spring Data JPA**: Higher-level Spring API that simplifies JPA-based data access through repositories.

Note 110

Common JPA Annotations and Entity Relationships

- **Mapping annotations**:
- **`@Entity`**: Marks a class as a persistent entity.
- **`@Table`**: Specifies the database table name.
- **`@Id`**: Marks the primary key.
- **`@GeneratedValue`**: Configures primary-key generation.
- **`@Column`**: Maps a field to a database column.
- **`@Transient`**: Excludes a field from persistence.
- **Relationship annotations**:
- **`@JoinColumn`**: Specifies a foreign-key column mapping.
- **`@OneToOne`**: Defines a one-to-one relationship.
- **`@OneToMany`**: Defines a one-to-many relationship.
- **`@ManyToOne`**: Defines a many-to-one relationship.
- **`@ManyToMany`**: Defines a many-to-many relationship.
- **Queries and cascade behavior**:
- **`@NamedQuery` and `@NamedQueries`**: Define one or multiple named queries for an entity.
- **`CascadeType.ALL`**: Propagates parent operations such as saving, updating, or deleting to child entities.

Note 111

Optimistic Locking with JPA

- **Optimistic locking**: Detects concurrent-update conflicts when data is committed or updated, rather than locking it during reads.
- **`@Version`**: Marks a version field used to prevent lost updates.
- **Update process**:

1. Read an entity with version `1`.
1. Modify the entity and attempt an update.
1. JPA updates only if the database version matches the in-memory version, then increments it.
1. If another update changed the version, no row matches and JPA throws `OptimisticLockException`.

- **Example:** `UPDATE account SET balance = 1000, version = 2 WHERE id = 201 AND version = 1;`

Note 112

Spring Data and JDBC Reference

- **Repository interfaces**:
- **`CrudRepository`**: Provides basic operations such as `save`, `findById`, and `deleteById`.
- **`JpaRepository`**: Extends `CrudRepository` with JPA-specific features, including batch operations.
- **`@EnableJpaRepositories`**: Enables scanning for Spring Data JPA repository interfaces.

- **Query results and mapping**:
- **Derived query**: A method such as `findByUsername()` derives its query from its name.
- **`findAll()` and `findById()`**: Return all records and one record wrapped in `Optional`, respectively.
- **`Pageable` and `Page<T>`**: Represent pagination settings and paginated results.
- **`@Query`**: Defines custom JPQL or native SQL.
- **JDBC utilities**:
- **`JdbcTemplate`**: Simplifies SQL execution, updates, and reading `ResultSet` data.
- **`RowMapper`**: Converts each `ResultSet` row into a Java object.
- **`NamedParameterJdbcTemplate`**: Supports named parameters in SQL.
- **`queryForObject()` and `query()`**: Retrieve one row and multiple rows, respectively.
- **Batch inserts**: Use `JdbcTemplate.batchUpdate()` or named-parameter batch operations.
- **Exception handling**: Spring translates SQL and JPA exceptions into the `DataAccessException` hierarchy.
- **Custom repositories**: Define a separate interface and implementation, then extend the custom interface from the main repository.

Note 113

Spring Web MVC Architecture and Request Flow

- **Spring MVC**: Spring web framework for applications using the Model–View–Controller pattern.
- **Architecture components**:
- **Model**: Holds application data and business logic; for example, a `User` class or service fetching user details.
- **View**: Presents data to the user, such as a JSP or HTML page displaying user details.
- **Controller**: Handles requests, interacts with the model, and chooses a view; for example, a `/getUser` handler calling a service.
- **`DispatcherServlet`**: Front controller that routes requests to the appropriate controllers and views.
- **Request flow**:

1. The user sends a request to `DispatcherServlet`.
1. `DispatcherServlet` consults `HandlerMapping` to find the controller.
1. The controller processes the request and returns `ModelAndView`.
1. `DispatcherServlet` consults `ViewResolver` to resolve the view.
1. The view renders the response and returns it to the user.

Note 114

Spring MVC Request Processing

- **Front controller**: `DispatcherServlet` receives incoming requests and coordinates their processing.
- **Controller selection**: `HandlerMapping` finds the handler and controller mapped to the request URL, then returns them to `DispatcherServlet`.
- **Controller execution**:
- `HandlerAdapter` invokes the controller's business logic.
- The controller places results in the `Model` and returns a logical view name.

- **View rendering**:
- `ViewResolver` maps the logical name to a `View`.
- `DispatcherServlet` asks the view to render the model data and return the response.

Note 115

DispatcherServlet Configuration and Lifecycle

- **XML configuration**: Define `DispatcherServlet` in `web.xml`.
- **Java configuration**: Use `@Configuration`, `@EnableWebMvc`, and implement `WebMvcConfigurer`.
- **Spring Boot**: `@EnableAutoConfiguration` handles the relevant configuration automatically.
- **Lifecycle**:
- Loaded through `web.xml` or `SpringBootServletInitializer`.
- Initialized during application startup when configured to load on startup.
- Delegates incoming requests to controllers.

Note 116

Spring MVC Controller Annotations

- **`@Controller`**: Marks a class as a Spring MVC controller, commonly used for web pages.
- **`@RestController`**: Combines `@Controller` and `@ResponseBody`; methods return response data directly.
- **Use**: REST APIs returning JSON, XML, or text.
- **Contrast**: Choose `@Controller` when returning views or controlling response handling individually.
- **`@RequestMapping`**: Maps web requests to controller classes or methods; supports HTTP methods and attributes such as `params` and `headers`.
- **Shortcut mappings**:
- **`@GetMapping`**: Equivalent to `@RequestMapping(method = RequestMethod.GET)`.
- **`@PostMapping`**: Equivalent to `@RequestMapping(method = RequestMethod.POST)`.

Note 117

Binding Request Data in Spring MVC

- **`@RequestParam`**: Binds a query parameter to a method parameter; common in GET requests, forms, and filtering.
- *Example:* `GET /greet?name=Kodewala` can produce `Hello, Kodewala`.
- **`@PathVariable`**: Binds a variable embedded in the URL path, often identifying a resource.
- *Example:* `GET /user/101` can produce `User ID: 101`.
- **Choosing between them**:
- **Path variable**: Use for a value that is part of the resource path, such as an ID.
- **Request parameter**: Use for optional or additional query information.
- **`@RequestBody`**: Binds an HTTP request body, such as JSON or XML, to a Java object; commonly used with POST and PUT.
- *Example:* JSON containing `name: Kodewala` and `age: 40` can be bound to an object.

- **Other binding options**:
- **`@ModelAttribute`**: Binds request parameters to a model object and makes it available to the view.
- **`@RequestHeader`**: Binds a named HTTP header to a method parameter.
- **`@InitBinder`**: Customizes request parameter binding through `WebDataBinder`.

Note 118

Returning Views and REST Responses

- **String return value**: Represents a logical view name when returning a page rather than JSON or plain data.
- *Example:* `ViewResolver` can map `home` to a JSP.
- **`ModelAndView`**: Combines a view name with model data in one return object.
- *Example:* Return `userView.jsp` with `name = "Suresh"` in the model.
- **`ResponseEntity`**: Supports REST responses with control over the response body, HTTP status, and headers.
- **Passing data to a view**: Add attributes using `Model`, `ModelMap`, or `ModelAndView`.
- **Returning JSON**: Use `@ResponseBody` or `@RestController`; Java objects can be converted automatically by Jackson.
- **Content negotiation**: Spring uses the `Accept` header and configured converters to select formats such as JSON or XML.

Note 119

Spring MVC Exception Handling

- **Local handling**: Use `@ExceptionHandler` inside a controller to handle exceptions from that controller.
- **Use**: Apply controller-specific logic, messages, or fine-grained error handling.
- **Global handling**: Combine `@ControllerAdvice` with `@ExceptionHandler` to centralize handling across controllers.
- *Example:* A global handler can handle `NullPointerException` across the application.
- **Purpose of `@ExceptionHandler`**: Identifies a method that handles exceptions thrown by controller methods.

Note 120

HTTP Methods and Their Uses

- **Retrieving and creating**:
- **GET**: Retrieves data, such as user details or product listings.
- **POST**: Creates data, such as registering a user or submitting a contact form.
- **Updating**:
- **PUT**: Fully replaces or updates an existing resource, such as an entire user profile.
- **PATCH**: Partially updates a resource, such as changing an email or order status.
- **Removing**:
- **DELETE**: Removes data, such as deleting an account or product.

- **Metadata and capabilities**:
- **HEAD**: Retrieves headers without the response body, such as checking file availability.
- **OPTIONS**: Identifies supported endpoint methods; commonly used for CORS preflight requests.

Note 121

HTTP Status Code Categories

- **2xx — Success**:
- **200 OK**: Request succeeded.
- **201 Created**: A resource was created.
- **204 No Content**: Request succeeded without a response body, often after DELETE.
- **3xx — Redirection**:
- **301 Moved Permanently**: Permanent URL redirect.
- **302 Found**: Temporary redirect.
- **304 Not Modified**: Supports caching and saves bandwidth.
- **4xx — Client errors**:
- **400 Bad Request**: Invalid request data or syntax.
- **401 Unauthorized**: Authentication is missing or invalid.
- **403 Forbidden**: Access is denied despite authentication.
- **404 Not Found**: The resource does not exist.
- **5xx — Server errors**:
- **500 Internal Server Error**: Generic server failure.
- **502 Bad Gateway**: Upstream server error.
- **503 Service Unavailable**: Server is down or overloaded.
- **504 Gateway Timeout**: Upstream response timed out.

Note 122

Additional Spring MVC Capabilities

- **CORS**: Enable cross-origin access using `@CrossOrigin` on a controller or method.
- **File uploads**: Accept `MultipartFile` parameters and configure a multipart resolver.
- **Configuration**: Set upload size limits and configure the `MultipartResolver` bean.
- **Validation**: Use `@Valid` on form objects and inspect `BindingResult` for errors.
- **Servlet request and response**: Declare `HttpServletRequest` or `HttpServletResponse` as controller method parameters.

Note 123

Spring Supporting Modules and Security Concepts

- **Supporting modules**:
- **Spring Security**: Protects applications using login and role-based access control.
- **AOP**: Separates cross-cutting concerns, such as logging and transaction management, from business logic.
- **Validation**: Checks that input satisfies rules; for example, verifies email format during

registration.
- **Cache**: Temporarily stores frequently accessed data to improve performance; for example, caches product details to reduce database load.
- **Authentication**: Verifies a user's identity, typically using a username and password.
- **Authorization**: Determines what an authenticated user may access.

Note 124

Servlet Filters and Spring Security Request Flow

- **Filter**: Servlet API component that intercepts requests and responses before they reach a servlet.
- **Functions**: Process, modify, or block HTTP requests and responses.
- **Security filter chain**: Spring Security uses ordered filters, each with a specific security responsibility, to process requests.
- **Request flow**:

1. Client sends a request through security filters.
1. The request reaches `DispatcherServlet` and then the controller.
1. The response returns through the filters to the client.

- **Key distinction**: Security filters run before the request reaches `DispatcherServlet`.

Note 125

Session-Based Authentication

- **Session-based authentication**: Default Spring Security mechanism; stores authentication in a server-side HTTP session.
- **Login and authentication flow**:

1. User submits credentials to `/login`.
1. `UsernamePasswordAuthenticationFilter` extracts credentials and creates an `UsernamePasswordAuthenticationToken`.
1. The filter calls `AuthenticationManager` to authenticate the token.
1. `DaoAuthenticationProvider` typically uses `UserDetailsService` and `PasswordEncoder`.
1. `UserDetailsService` loads a `UserDetails` object containing username, password, and roles.
1. `PasswordEncoder` checks the submitted password against the stored password.

- **`{noop}`**: Plain-text comparison; real applications should use an encoder such as `BCryptPasswordEncoder`.

1. The provider returns an authenticated token or throws an exception on failure.
1. The filter continues on success or redirects to the login page with an error on failure.
1. The authenticated result is stored in `SecurityContextHolder` and persisted in the HTTP session.

- **Subsequent requests**:
- `SecurityContextPersistenceFilter` loads the context from the session.
- The user remains authenticated until logout or session expiry.
- Authorization is checked on every request before access is allowed.

Note 126

JWT-Based Authentication

- **JWT authentication**: Stateless mechanism that returns a token after successful login; the server does not rely on an HTTP session.
- **Login flow**:

1. User sends username and password to `/login`.
1. `JwtAuthenticationFilter` extracts credentials and calls `AuthenticationManager`.
1. `DaoAuthenticationProvider` loads the user through `UserDetailsService` and verifies the password.
1. A valid login produces a JWT returned to the client.

- **Subsequent requests**:

1. Client stores the token, for example in localStorage or sessionStorage.
1. Client sends it in `Authorization: Bearer <token>`.
1. `JwtAuthorizationFilter` validates the token and sets the user in `SecurityContext`.
1. Spring checks roles and allows or denies access.

- **Typical uses**: REST APIs, mobile apps, stateless backends, and microservices sharing tokens.

Note 127

OAuth 2.0 Third-Party Authentication

- **OAuth 2.0**: Token-based authorization framework that lets a third-party application access user resources without exposing the user's password.
- **Common uses**: Login with Google, GitHub, or Facebook; delegated access to resources such as Dropbox files.
- **Token exchange flow**:

1. User selects login and is redirected to an identity provider, such as Google.
1. User authenticates and consents; the provider redirects back with an authorization code.
1. Spring exchanges the code for an access token and, optionally, an ID token.
1. Spring uses the access token to retrieve user information.
1. User information is wrapped in `DefaultOAuth2User` and stored in `SecurityContext`.
1. The authenticated user is redirected to a protected resource, such as `/home`.

Note 128

Choosing an Authentication Mechanism

- **Session-based authentication**:
- **Best suited to**: Traditional Spring MVC or JSP applications, especially small-to-medium applications using server-managed sessions.
- **JWT authentication**:
- **Best suited to**: REST APIs, mobile apps, and stateless microservices.
- **Benefits**: Supports scalable backends and token sharing across services.
- **OAuth 2.0**:
- **Best suited to**: Third-party login and delegated access to user resources.
- **Examples**: Google or Facebook login; access to Dropbox files.
- **E-commerce example**: Use sessions for a server-managed web login, JWT for mobile apps and microservices, and OAuth 2.0 for third-party login.

Note 129

Spring Security Annotations

- **Configuration annotations**:
- **`@EnableWebSecurity`**: Enables web security configuration.
- **`@EnableMethodSecurity`**: Enables method-level security.
- **Method authorization**:
- **`@PreAuthorize`**: Applies a flexible check before execution; can access method parameters.
- **`@PostAuthorize`**: Applies an authorization check after execution, including checks based on the return value.
- **Role-check annotations**:
- **`@Secured`**: Spring-specific, simple role check before method execution; less flexible.
- **`@RolesAllowed`**: JSR-250 standard for simple role checks before method execution.

Note 130

JWT Authentication and Spring Security Concepts

- **JWT request flow**:
1. **Login**: Generate a token after the user authenticates.
1. **Request validation**: Validate the token on each request.
1. **Security context**: Store the authenticated user in `SecurityContext` for that request.
- **JWT vs. sessions**:
- **JWT**: Stateless; well suited to distributed microservices.
- **Session-based authentication**: Requires shared or centralized session storage in distributed systems.
- **Method security annotations**:
- **`@PreAuthorize`**: Supports SpEL; preferred for fine-grained access control.
- **`@Secured` and `@RolesAllowed`**: Provide role-based access control.
- **OAuth2 login**: Configure an identity provider such as Google or GitHub, then handle the user details returned after login.
- **CSRF protection**:
- **CSRF**: Cross-site request forgery, where a user's authenticated browser is tricked into making an unwanted request.
- **Spring protection**: Uses CSRF tokens to verify requests.
- **When to disable**: Stateless APIs that do not rely on browser-managed credentials may disable CSRF protection.
- **Security context**:
- **`SecurityContext`**: Holds the authenticated user's security details for a request.
- **`SecurityContextHolder`**: Provides access to the current security context.
- **Custom authentication**:
- **`AuthenticationProvider`**: Implement custom authentication and credential validation.
- **`UserDetailsService`**: Load user details for validation against a database or external service.

Note 131

Spring AOP and Cross-Cutting Concerns

- **AOP**: Aspect-Oriented Programming modularizes concerns that affect multiple classes, such as logging, security, transactions, auditing, and caching.
- **Purpose and benefits**:
- **Separation of concerns**: Keeps business logic focused on core functionality.
- **Less duplication**: Define repeated behavior once and apply it across methods.
- **Modularity**: Encapsulates each cross-cutting concern in an aspect.
- **Maintainability**: Supports cleaner, more testable code; for example, `deleteUser` can focus on business rules while logging and security are applied separately.
- **Creating and applying an aspect**:

1. **Add dependencies**: Include the required Maven dependencies.
1. **Create an aspect class**: Define the cross-cutting behavior.
1. **Enable AOP**: Configure Spring AOP or use `@EnableAspectJAutoProxy`.

- **Common annotations**:
- **`@Aspect`**: Marks a class as an aspect.
- **`@Before`**: Runs before a matched method.
- **`@After`**: Runs after a matched method finishes.
- **`@AfterReturning`**: Runs after a matched method returns successfully.
- **`@Around`**: Runs before and after a matched method; can control whether it executes.
- **`@Pointcut`**: Defines a reusable expression for matching method executions, or join points.

Note 132

Spring Validation and Binding Results

- **Spring Validation**: Checks Java objects, such as form beans, for data integrity before processing; commonly used for Spring MVC input.
- **Implementation sequence**:

1. **Add dependencies**: Include the required validation libraries.
1. **Annotate the bean**: Add constraints such as `@NotNull` or `@Size`.
1. **Trigger validation**: Use `@Valid` on an input parameter.

- **Validation tools**:
- **`@Valid`**: Tells Spring to validate an object using its constraint annotations; commonly placed on a model attribute or request body.
- **`Validator`**: Spring interface for custom validation when annotations are insufficient.
- **`@Validated`**: Spring validation annotation used to trigger validation, including support for validation groups.
- **`BindingResult`**: Holds validation errors and must immediately follow the corresponding `@Valid` parameter.

Note 133

Caching Concepts, Strategies, and Spring Cache

- **Cache**: Fast-access storage for frequently requested data, avoiding repeated reads from

primary storage such as a database, API, or filesystem.
- **Good use cases**:
- **Frequently requested, slowly changing data**: Product details, weather data, exchange rates, country codes, and language mappings.
- **Repeated lookups**: Session or authentication data, when caching is appropriate for its security and consistency needs.
- **Benefits**:
- **Performance**: Memory-based retrieval reduces latency and improves response times.
- **Scalability**: Handles increased demand without proportional growth in backend resources.
- **Cost efficiency**: Reduces database compute and read costs.
- **Cache types**:
- **Client-side**: Browser or local application memory.
- **Server-side**: Backend application cache, such as in-memory storage or Redis.
- **CDN**: Static content stored at edge locations, such as Cloudflare.
- **Database cache**: Frequently used query results, often stored in Redis.
- **Caching strategies**:
- **Cache-aside (lazy loading)**: Load on demand; on a miss, fetch from the database and populate the cache. Suited to read-heavy applications.
- **Write-through**: Write to cache and backing store together; improves consistency but slows writes.
- **Write-behind (write-back)**: Write to cache first and update the backing store asynchronously; fast, but cache failure can risk data loss.
- **Read-through**: Application uses the cache, which fetches and stores data after a miss.
- **Key terms**:
- **Cache hit**: Requested data is found in the cache and returned quickly.
- **Cache miss**: Data is absent, so it is fetched from the source and may then be cached.
- **TTL**: Time to live; duration before cached data expires.
- **LRU / LFU**: Least Recently Used / Least Frequently Used cache replacement approaches.
- **When not to cache**: Frequently changing data, sensitive data requiring strict consistency, or cases with limited memory and little performance benefit.
- **Best practices**: Choose the appropriate cache layer, use TTLs to limit stale data, and monitor hit/miss ratios.
- **Spring Cache abstraction**:
- **Purpose**: Reduces calls to expensive methods, such as database access.
- `**@EnableCaching**`: Enables annotation-driven cache management.
- `**@Cacheable**`: Caches a method's result; returns a cached value when available, otherwise runs the method and stores its result.
- `**@CachePut**`: Updates the cache while allowing the method to execute.
- `**@CacheEvict**`: Removes cache entries.
- `**@Caching**`: Groups multiple cache annotations.
- `**@CacheConfig**`: Shares cache settings at class level.

Note 134

Spring Boot Features and Framework Comparison

- **Spring Boot**: Extension of Spring that simplifies application setup, configuration, and deployment.
- **Key features**:
- **Auto-configuration**: Configures the application based on classpath libraries.
- **Starters and dependency management**: Bundles common libraries and helps manage

versions.
- **Embedded servers**: Runs as a standalone application, with Tomcat as the default server.
- **Production readiness**: Provides monitoring, metrics, and health checks through Actuator.
- **Opinionated defaults**: Supplies sensible defaults while allowing customization.
- **Spring Framework vs. Spring Boot**:
- **Spring Framework**: Flexible, powerful, and highly customizable, but often requires more manual setup and boilerplate.
- **Spring Boot**: Built on Spring; reduces configuration and speeds development through defaults, auto-configuration, and embedded servers.
- *Example:* A Spring project may require manual database and Tomcat configuration; Spring Boot can create a REST API with `spring-boot-starter-web` and no external server setup.
- **Changing the embedded server**: Replace Tomcat with a server such as Jetty by excluding the Tomcat dependency and adding Jetty.
- **Convention over configuration**: Spring Boot applies sensible defaults and conventions so developers can omit configuration unless they need to override it.
- **Component scanning**: A main class in the root package enables scanning of subpackages.
- **Data source**: A database dependency can allow Spring Boot to auto-configure a data source.
- **Spring Boot starters**: Maven or Gradle dependencies that aggregate libraries for a specific purpose, reducing manual setup, version conflicts, and development time.
- **Web**: `spring-boot-starter-web` for Spring MVC.
- **Database**: `spring-boot-starter-data-jpa`, `spring-boot-starter-jdbc`, and `spring-boot-starter-data-mongodb`.
- **Security and views**: `spring-boot-starter-security` and `spring-boot-starter-thymeleaf`.
- **Testing**: `spring-boot-starter-test` includes common tools such as JUnit and Mockito.

Note 135

Spring Boot Application Setup and Configuration

- **`@SpringBootApplication`**: Main application annotation combining three capabilities:
- **`@SpringBootConfiguration`**: Marks the class as a source of bean definitions.
- **`@EnableAutoConfiguration`**: Enables configuration based on the classpath and environment.
- **`@ComponentScan`**: Scans the main class's package and subpackages for components.
- **Component scanning choices**:
- **Default practice**: Place the main class in the root package so subpackages are scanned.
- **Exclusions**: Exclude unwanted packages or components to avoid loading test/mock or legacy classes and to reduce startup time and memory use.
- **Project generation**:
- **Spring Initializr**: Web-based project scaffolding tool at `https://start.spring.io`.
- **Benefits**: Generates Maven or Gradle projects quickly, with appropriate structure and dependencies.
- **Externalized configuration**:
- **Default files**: `application.properties` or `application.yml` in `src/main/resources/`.
- **Purpose**: Configure settings such as server port, data source, logging, and custom properties.
- **Profiles**:
- **Purpose**: Separate configuration and beans by environment, such as `dev`, `test`, and `prod`.
- **Files**: Use names such as `application-dev.properties` and `application-prod.properties`.

- **Activation options**: Set the active profile in configuration, pass a command-line argument, or set an environment variable.
- **Best practice**: Keep shared defaults in the base file and override only environment-specific settings, such as database URLs, logging levels, and ports.

Note 136

Spring Boot Actuator and Testing

- **Spring Boot Actuator**: Provides production-ready endpoints for application monitoring and management.
- **Common endpoints and integrations**:
- **Health**: `/actuator/health`.
- **Metrics**: `/actuator/metrics`.
- **Application information**: `/actuator/info`.
- **Environment details**: `/actuator/env`.
- **Monitoring tools**: Integrates with Prometheus, Grafana, and Spring Boot Admin.
- **Setup and security**:
- **Dependency**: Add `spring-boot-starter-actuator`.
- **Access control**: Secure endpoints with Spring Security rules or management security properties.
- **Spring Boot test annotations**:
- **`@SpringBootTest`**: Full integration testing.
- **`@WebMvcTest`**: Tests the Spring MVC controller layer.
- **`@DataJpaTest`**: Tests JPA repositories in isolation, commonly with H2.
- **`@MockBean`**: Adds a mock to the Spring Boot test context.

Note 137

Spring Boot Interview Review

- **Running an application**: Use `SpringApplication.run(Application.class, args)` or run the generated JAR.
- **Data access**:
- **Database connection**: Configure data source properties and use Spring Data JPA or JDBC.
- **Spring Data JPA**: Simplifies database access through JPA-based repositories.
- **Starter parent**: Parent POM that manages dependency versions and plugin configuration.
- **Application behavior**:
- **Startup code**: Implement `CommandLineRunner` or `ApplicationRunner`.
- **Dependency injection**: Use constructor, setter, or field injection; constructor injection is commonly preferred.
- **Global exceptions**: Use `@ControllerAdvice` with `@ExceptionHandler`.
- **Stereotype annotations**:
- **`@Component`, `@Service`, `@Repository`, `@Controller`**: Identify bean roles; `@Repository` supports persistence exception translation, while `@Controller` handles web requests.
- **Web endpoints**:
- **REST service**: Use `@RestController` and mappings such as `@GetMapping` and `@PostMapping`.

- **`@RestController` vs. `@Controller`**: `@RestController` combines `@Controller` and `@ResponseBody`, serializing return values, commonly as JSON or XML.
- **Configuration and security**:
- **`@EnableAutoConfiguration`**: Enables auto-configuration and is normally included through `@SpringBootApplication`.
- **External settings**: Use properties or YAML files, environment variables, or command-line arguments.
- **Profiles**: Separate configuration and beans for environments such as development, testing, and production.

Note 138

Monolithic Architecture

- **Definition**: Traditional approach where an application's components are built, packaged, and deployed as one unit.
- **Layers**:
- **Presentation**: Manages the user interface and interactions.
- **Business logic**: Contains processing logic, rules, and computations.
- **Data access**: Handles persistence and database interactions.
- **Drawbacks**:
- **Single point of failure**: A module failure can bring down the entire application.
- **Technology lock-in**: The application is usually tied to one technology stack.
- **Tight coupling**: Changes can affect other components and increase regression risk.
- **Deployment and maintenance**: Even small changes require rebuilding and redeploying; growing codebases become harder to understand and test.
- **Resource limits**: Components share server resources, and the application must scale as a whole.

Note 139

Microservices Architecture

- **Definition**: Architectural design pattern that structures an application as small, independent, loosely coupled services.
- **Not a technology**: Microservices is not a programming language, framework, or API; it can use any technology stack.
- **Advantages**:
- **Independent work**: Services can be developed, tested, deployed, and scaled separately.
- **Resilience**: Failure in one service need not stop the entire system.
- **Technology choice**: Services can use different languages or databases.
- **Delivery and operations**: Focused codebases simplify maintenance; teams can release in parallel and distribute load.
- **Challenges**:
- **Bounded context**: Decide how many services are needed and which functionality belongs in each.
- **Repeated configuration**: Shared settings, such as data sources, Kafka, or Redis, may be duplicated.
- **Limited visibility**: Distributed ownership can make it harder to understand the whole

application.
- **Architecture flexibility**: No fixed design is required; supporting components are selected to meet project needs.

Note 140

Microservices Supporting Components

- **Optional components**: Select components according to project requirements; none is mandatory in every setup.
- **Routing and discovery**:
- **API gateway**: Entry point that handles routing, filtering, security, and rate limits. *Example:* Spring Cloud Gateway, Netflix Zuul.
- **Service registry**: Tracks service instances and helps services discover one another without hardcoded URLs. *Example:* Eureka, Consul, ZooKeeper.
- **Load balancer**: Distributes traffic across instances to improve availability and scalability. *Example:* Spring Cloud LoadBalancer, Nginx.
- **Configuration and communication**:
- **Config server**: Centralizes external and environment-specific configuration. *Example:* Spring Cloud Config Server.
- **Feign client**: Simplifies inter-service REST calls and reduces boilerplate. *Example:* Spring Cloud OpenFeign.
- **Kafka server/message broker**: Enables asynchronous publish-subscribe communication. *Example:* Apache Kafka, RabbitMQ.
- **Reliability and security**:
- **Circuit breaker**: Stops calls to repeatedly failing services. *Example:* Resilience4j, Hystrix.
- **Security server**: Provides authentication, authorization, roles, and token support. *Example:* Spring Security, Keycloak, Okta.
- **Redis cache**: Stores frequently used key-value data to reduce database calls. *Example:* Redis.
- **Operations and visibility**:
- **Admin server**: Monitors services and exposes health, metrics, and environment information. *Example:* Spring Boot Admin.
- **Distributed tracing**: Follows requests across services to find bottlenecks. *Example:* Zipkin, Jaeger.
- **Monitoring and dashboards**: Prometheus collects metrics; Grafana displays them.
- **Logging and documentation**: ELK centralizes logs; Swagger UI or Springdoc OpenAPI generates and visualizes API documentation.

Note 141

Choosing and Migrating Architecture

Choosing
- **Application size**:
- **Monolithic**: Suits small, simple applications and prototypes with minimal setup.
- **Microservices**: Suits large, complex systems needing modularity and scalability.
- **Team and infrastructure**:
- **Monolithic**: Requires general development skills and can run on one server.

- **Microservices**: Requires cloud, API, container, and distributed-systems expertise; needs robust infrastructure and tooling.

Migration

- **Assess and decompose**:

1. **Assess the monolith**: Map modules, dependencies, bottlenecks, and tightly coupled areas.
1. **Define business capabilities**: Decompose by business domains or bounded contexts; map each service to one capability, such as billing or inventory.

- **Prepare and extract**:

1. **Establish infrastructure**: Set up gateway, discovery, configuration, monitoring, logging, CI/CD, and containers.
1. **Strangle the monolith**: Extract one module at a time; route selected requests to new services while the rest remains intact.
1. **Build communication**: Use REST or message queues; design robust APIs and handle failures with circuit breakers.

- **Secure and validate**:

1. **Migrate data carefully**: Prefer one database per service; consider Event Sourcing or Change Data Capture; avoid shared databases.
1. **Refactor security**: Centralize authentication and authorization; use OAuth2 or token-based API security such as JWT.
1. **Test extensively**: Run integration and end-to-end tests; verify communication, failover, and data consistency.

- **Complete transition**:

1. **Monitor and optimize**: Track performance, logs, metrics, and failures; tune deployments and architecture.
1. **Decommission the monolith**: Retire it after all services are migrated and stable.

Note 142

Apache Kafka: Purpose and Performance

- **Purpose**: Reliable messaging system originally built by LinkedIn, using publish-subscribe to send and receive data efficiently and durably.
- **Messaging model**:
- **Multiple consumers**: Producers and consumers can work concurrently; reading a message does not make it unavailable to other consumers.
- **Disk retention**: Messages remain on disk for later processing rather than requiring immediate consumption.
- **Key strengths**:
- **Throughput and latency**: Designed for high-volume, low-latency messaging; the material cites up to two million requests per second.
- **Scalability**: Capacity can be added while the system runs, without downtime.
- **High-load performance**: Maintains strong performance under heavy workloads.
- **Why Kafka is fast**:
- **Zero-copy**: Uses operating-system-level data transfer to reduce application-layer copying.
- **Batching**: Sends data in chunks, reducing overhead and cross-machine latency.
- **Sequential disk access**: Avoids random access and can approach memory-like speeds.
- **Horizontal scaling**: Spreads many topic partitions across machines to handle large loads.

Note 143

Kafka Use Cases and Core Concepts

- **Use cases**:
- **Activity tracking**: Captures user activity, Kafka's original LinkedIn use case.
- **Messaging**: One application produces notifications that another application reads and handles consistently.
- **Metrics and logging**: Streams operational data for collection and analysis.
- **Commit log**: Publishes database changes so applications can receive live updates.
- **Stream processing**: Handles and processes large streaming datasets.
- **Broker**: Message-broker intermediary that connects parties that may not know one another.
- **Log**: Append-only, time-ordered sequence of records; Kafka appends incoming records to it.
- **Topic**: Named log grouping related messages, comparable to a database table or filesystem folder. *Examples:* `orders`, `payments`, `delivery`.
- **Partition**:
- **Purpose**: Divides a topic to manage message load, increase throughput, and support redundancy and scalability.
- **Distribution**: Partitions can reside on different servers, letting a topic scale beyond one server's disk capacity.
- **Default noted**: The material gives 10 as the default partition count.
- **Storage setting**: `log.dir` specifies where Kafka stores log data on disk.

Note 144

Kafka Keys, Producers, and Message Sending

- **Key-based partitioning**:
- **Partition choice**: A keyed message is assigned using `hashCode(key) % number_of_partitions`.
- **Ordering**: Messages with the same key go to the same partition and remain ordered there.
- **Unkeyed messages**: Producers generally balance messages across a topic's partitions.
- **Producer**: Creates messages for a specified topic; producers do not need to manage partitions unless directing messages by key or explicit partition.
- **Batching and state**:
- **Batch**: Collection of messages produced to the same topic and partition.
- **Kafka state**: Kafka stores and retrieves topic messages but does not maintain producer or consumer state.
- **Sending flow**:

1. **Create record**: Specify a topic and value; optionally include a key or partition.
1. **Serialize**: Convert key and value to byte arrays for network transfer.
1. **Select partition**: Use the specified partition, or let the partitioner choose one.
1. **Batch and send**: Add the record to a batch for that topic and partition, then send it to the broker.
1. **Handle response**: On success, receive metadata containing topic, partition, and offset; on failure, the producer may retry before returning an error.

Note 145

Kafka Consumers, Offsets, and Consumer Groups

- **Consumer**: Subscribes to one or more topics and reads messages in production order within partitions.
- **Offset**: Integer position used to track which messages a consumer has processed.
- **Consumer group**: Coordinates consumers that read and process data in parallel.
- **Partition assignment**:
- **Single active consumer**: A partition is consumed by only one consumer in a group at a time.
- **Multiple partitions**: One consumer can handle multiple partitions; separate partitions can be processed concurrently.
- **Idle consumers**: If a group has more consumers than the topic has partitions, some consumers receive no messages.
- **Separate applications**: Create a distinct consumer group for each application that needs its own copy of topic messages.
- **Parallelism limit**: A single partition cannot be read concurrently by multiple consumers in the same group; more partitions enable more group-level parallelism.

Note 146

Kafka Reliability, Ordering, and Operations

- **Replication and availability**:
- **ISR**: In-sync replicas are replicas caught up with the leader; the interview hint links ISR acknowledgments to durability.
- **Broker failure**: Partitions can be re-elected on ISR replicas; availability depends on replication factor and ISR.
- **Acknowledgments**: `acks=0` waits for no acknowledgment, `acks=1` waits for the leader, and `acks=all` waits for all ISR replicas.
- **Delivery guarantees**:
- **Exactly-once semantics**: Aim to prevent duplicates and message loss using transactions, idempotent producers, and broker configuration.
- **Avoiding loss or duplication**: Combine idempotent producers, transactions, `acks=all`, and careful offset management.
- **Committed offsets**: The `__consumer_offsets` topic stores consumer-group offsets for recovery.
- **Ordering and partitioning**:
- **Ordering scope**: Kafka guarantees message ordering within a partition, not across a whole topic.
- **Key strategy**: Use a consistent key to keep related messages in the same ordered partition.
- **Partition count**: Affects ordering strategy, consumer parallelism, and performance.
- **Scaling trade-offs**:
- **More partitions**: Increase parallelism and potential throughput.
- **Too many partitions**: Increase metadata and broker-memory overhead, leadership-election time, and—in the material's interview hint—ZooKeeper load.
- **Consumer-group rebalancing**: Occurs when group membership or topic partitions change; cooperative rebalancing can reduce disruption.
- **Monitoring and tuning**: Use JMX metrics or Prometheus/Grafana; track lag, throughput, garbage collection, and disk I/O; tune batch size, `linger.ms`, and fetch size.

Note 147

Kafka Ecosystem and Schema Compatibility

- **ZooKeeper role**: Manages cluster metadata and controller election in the described Kafka setup.
- **Kafka Connect**: ETL integration tool for connecting Kafka with external systems.
- **Kafka Streams**: Library for building stream processing into an application.
- **Schema evolution**:
- **Formats**: Use Avro or Protobuf with a Schema Registry.
- **Compatibility practices**: Follow schema-evolution rules, such as adding optional fields and avoiding removal of required fields.

Note 148

Microservices Architecture and Principles

- **Microservices**: small, self-contained services organized around business domains and deployed independently.
- **Design basis**: Domain-Driven Design models services around bounded contexts for specific business domains.
- **Communication**: services run in separate processes and communicate through lightweight mechanisms, often HTTP resource APIs.
- **Benefits**:
- **Parallel development**: teams can work on independent modules.
- **Faster delivery**: changes affect a narrower domain, supporting quicker releases and CI/CD.
- **Deployment flexibility**: components can be distributed across clouds and data centers.
- **Core principles**:
- **Scalability** and **flexibility**.
- **Autonomy** and **decentralized governance**.
- **Resiliency** and **failure isolation**.
- **Continuous delivery** through DevOps.

Note 149

Database per Microservice

- **Pattern**: each Microservice owns its database, enabling independent development and deployment.
- **Data access**: other services cannot access a service's database directly; persistent data is exposed through APIs.
- **Benefits**:
- **Independent change and scaling**: database changes and capacity adjustments need not affect other services.
- **Encapsulation**: domain data remains within its owning service.
- **Failure isolation**: one database outage need not affect other services.
- **Polyglot persistence**: choose storage suited to each service's data and workload.
- *Example:* Product catalog uses a NoSQL document database for high-volume reads and JSON objects.

- *Example:* Shopping cart uses a distributed cache with a simple key-value store.
- *Example:* Ordering uses a relational database for rich relationships.

Note 150

CQRS and Read-Write Separation

- **CQRS**: Command and Query Responsibility Segregation; separates database writes from reads.
- **Why separate**:
- **Complex reads**: joins across many tables can cause latency and database locking.
- **Complex writes**: validations and lengthy business logic can also slow or lock operations.
- **Workload fit**: read-intensive applications can optimize read and write resources independently.
- **Responsibilities**:
- **Commands**: create or update data; may be sent through message brokers for asynchronous processing.
- *Example:* “Add the item to the bucket” or “Check my balance.”
- **Queries**: retrieve data without modifying the database; commonly return JSON through DTO objects.
- **Synchronizing databases**:
- **Event-driven updates**: a write-side update publishes an event; the read side consumes it and applies the latest changes.
- **Eventual consistency**: read data may lag until synchronization completes, then converges if updates stop.
- **Independent scaling**: read and write databases can be scaled separately; read replicas can seed the read database.

Note 151

Event Sourcing

- **Event Sourcing**: stores a sequence of events representing changes, rather than only the entity’s latest state.
- **Event store**:
- **Single source of truth**: events are saved in sequence in the write database.
- **Replay**: events can be replayed after failures to rebuild state and restore consistency.
- **History**: timestamped events make it possible to reconstruct data at a chosen point in time.
- **Read models**:
- **Materialized views**: combine and summarize events into denormalized data for querying.
- **Event distribution**: publish/subscribe messaging can deliver events to processes that build read views.
- **Example**: instead of overwriting a user’s details, record each update as an event; for a shopping cart, append every user action and summarize them in a read view.
- **Possible event-store technologies**: Azure Cosmos DB, Cassandra, and Event Store databases.

Note 152

Saga for Distributed Transactions

- **Saga**: sequence of local transactions that maintains consistency across services without one global transaction.
- **Transaction flow**:
- Each local transaction updates its service database and publishes a message or event to trigger the next transaction.
- If a business rule causes failure, compensating transactions reverse earlier changes.
- **Implementation styles**:
- **Choreography**: services publish domain events that trigger transactions in other services.
- *Example:* Order service creates an order and publishes “order created”; customer service checks and reserves credit, then emits “credit reserved” or “credit limit exceeded.”
- **Trade-off**: works well with fewer services, but event handling becomes complex as services grow.
- **Orchestration**: a central orchestrator directs local transactions and tracks overall status.
- *Example:* orchestrator creates a pending order, requests credit reservation, then advances the order or initiates rollback based on the result.
- **Trade-off**: suits complex workflows and future service additions; reduces cyclical dependencies.
- **Use Saga when**:
- Data consistency is needed across distributed services without tight coupling.
- Failed steps must trigger rollback or compensation.

Note 153

Backend for Frontend (BFF)

- **BFF**: an intermediate layer that aggregates and transforms backend-service responses for a particular frontend.
- **Purpose**:
- Reduces frontend work for combining and formatting data from multiple services.
- Provides responses tailored to the UI, minimizing frontend logic and rendering burden.
- **Organization**:
- A BFF usually serves one UI and is often managed by the frontend team.
- Multiple BFFs can serve different clients using the same backend.
- *Example:* separate BFFs for web and mobile when each displays data differently.
- **Best fit**: complex screens requiring processed data from multiple services or third-party integrations, especially when optimized rendering matters.
- **When unnecessary**: simple applications where each service has its own UI; avoid needless duplication when implementing BFFs.

Note 154

API Gateway

- **API Gateway**: reverse proxy and single client entry point that routes requests to underlying services while hiding internal architecture.

- **Cross-cutting functions**: can provide authentication, SSL termination, and caching.
- **Risks and safeguards**:
- **Single point of failure**: one gateway node can make the system vulnerable; deploy redundantly where needed.
- **Overloaded responsibilities**: adding excessive complex logic can turn the gateway into an anti-pattern.
- **Deployment choice**: consider multiple gateways for different services, based on client business context; avoid assuming one gateway should serve all internal services.

Note 155

Strangler Pattern for Migration

- **Strangler Pattern**: gradually replaces a large monolithic application with Microservices when a full migration at once is impractical.
- **Migration sequence**:

1. Identify monolithic functionality organized around a business domain.
1. Build Microservices to handle the selected functionality.
1. Route a portion of traffic to the new services through a gateway or proxy.
1. Gradually increase Microservice traffic and reduce traffic to the monolith.
1. Monitor performance and adjust traffic allocation.
1. Route all traffic to Microservices and retire the monolith.

Note 156

Circuit Breaker Pattern

- **Circuit Breaker**: monitors calls between services and stops repeated requests after failures reach a configured threshold.
- **When tripped**: further calls are skipped and return an error or another configured response.
- **Failure handling**: may return a default value or cached response to degrade gracefully.
- **Implementation steps**:

1. Identify communicating Microservices.
1. Place a circuit breaker between them.
1. Configure status monitoring and failure detection.
1. Block further requests when a service fails to respond and apply the chosen fallback.

Note 157

Externalized Configuration

- **Externalized configuration**: keep service configuration outside the service package so deployed instances can share configuration.
- **Why it matters**: Microservices may run in many independently deployed and scaled container instances.
- **Packaged-configuration risk**: changing configuration may require redeploying every instance, leaving some instances on old settings and creating inconsistency.
- **Recommended approach**: use shared external configuration to update service settings

consistently across instances.

Note 158

External Configuration Management

- **Configuration storage**: settings are kept outside the Microservice.
- **Sources**: databases, file systems, or environment variables.
- **Startup behavior**: the deployed service loads configuration from its external source.
- **Runtime updates**: changed settings are generally reloaded without deploying the service again.

Note 159

Consumer-Driven Contract Testing

- **Definition**: test-driven, design-first approach in which consumers define expected provider responses.
- **Contract ownership**: consumer developers write contracts describing expected responses to service requests.
- **Format**: typically a JSON or XML file accessible to both services and their developers.
- **Benefits**:
- **Early collaboration**: providers and consumers negotiate expectations from the start.
- **Clear requirements**: providers learn consumer needs proactively.
- **Fewer integration issues**: agreeing on the contract early reduces mismatches and testing cycles.
- **Implementation steps**:

1. **Define the contract**: consumer specifies expected inputs, outputs, and other requirements.
1. **Implement the contract**: provider builds the service to meet those requirements.
1. **Run tests**: provider checks compliance using unit and integration tests.
1. **Publish the contract**: provider shares it with the consumer after implementation and successful tests.
1. **Verify compatibility**: consumer confirms the service matches the agreement.

Note 160

Purpose of Microservice Patterns

- **Common challenges**: Microservices architectures can make scalability management and distributed transactions more complex.
- **Pattern value**: repeatedly tested patterns offer solutions to recurring architectural problems.
- **Design insight**: understanding patterns clarifies how Microservices support performance, scalability, agility, and maintainability.